

Sequence to Sequence Learning with Neural Networks

用神经网络进行序列到序列学习

Ilya Sutskever Oriol Vinyals Quoc V. Le

Google

{ilyasu, vinyals, qvl}@google.com

伊利亚·苏茨克维尔·奥里奥尔·维尼亚尔斯·夸克·V·黎(均来自 Google)

Advances in Neural Information Processing Systems 27 (NIPS 2014), pp. 3104–3112

Abstract

摘要

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.

深度神经网络 (DNN) 是非常强大的模型,在许多困难的机器学习任务上都取得了卓越的表现。虽然只要有足够大的有标注训练集,DNN 通常就能工作得很好,但它无法被直接用来完成“序列到序列”的映射。本文提出了一种对序列结构几乎不做假设的、端到端的通用序列学习方法。我们的方法使用一个多层长短期记忆网络 (LSTM) 将输入序列映射成一个固定维度的向量,然后用另一个深度 LSTM 从该向量“解码”出目标序列。主要实验结果为:在 WMT'14 英语—法语翻译任务上,由 LSTM 生成的译文在整个测试集上达到了 34.8 的 BLEU 分数,其中由于词表外的词对分数造成了一定的惩罚。值得一提的是,LSTM 在长句上并未遇到困难。作为对比,基于短语的传统统计机器翻译 (SMT) 系统在同一数据集上的 BLEU 分数为 33.3。当我们使用 LSTM 对上述 SMT 系统所产生的 1000 条候选翻译进行重排序 (re-rank) 时,其 BLEU 分数可以提升到 36.5,非常接近当时该任务的最佳公开结果。LSTM 还学到了对词序敏感、对主动/被动语态相对稳健的短语级与句子级表示。最后,我们发现:在所有源语句(但不改变目标语句)中反转词序可以显著提升 LSTM 的性能,这是因为反转词序在源语言和目标语言之间引入了许多“短期依赖”,从而大大降低了优化问题的难度。

1. Introduction

引言

Deep Neural Networks (DNNs) are extremely powerful machine learning models that achieve excellent performance on difficult problems such as speech recognition and visual object recognition. DNNs are powerful because they can perform arbitrary parallel computation for a modest number of steps. A surprising example of the power of DNNs is their ability to sort N N -bit numbers using only 2 hidden layers of quadratic size. So, while neural networks are related to conventional statistical models, they learn an intricate computation. Furthermore, large DNNs can be trained with supervised backpropagation whenever the labeled training set has enough information to specify the network's parameters. Thus, if there exists a parameter setting of a large DNN that achieves good results (for example, because humans can solve the task very rapidly), supervised backpropagation will find these parameters and solve the problem.

深度神经网络 (DNN) 是极为强大的机器学习模型,在诸如语音识别、视觉目标识别等困难问题上都能取得优异的表现。DNN 之所以强大,是因为它能够在很少的几步内完成任意的并行计算。一个令人惊讶的例子是:DNN 仅用两个二次大小的隐藏层就能完成对 N 个 N 位整数的排序。因此,虽然神经网络与传统的统计模型有关联,但它学到的是一种复杂精巧的计算过程。此外,只要有标注训练集所提供的信息足以确定网络的参数,大型 DNN 就可以通过有监督的反向传播进行训练。也就是说,如果存在一组大型 DNN 的参数能够取得良好效果(比如因为人类能很快完成这个任务),那么有监督的反向传播就会找到这组参数,从而解决该问题。

Despite their flexibility and power, DNNs can only be applied to problems whose inputs and targets can be sensibly encoded with vectors of fixed dimensionality. It is a significant limitation, since many important problems are best expressed with sequences whose lengths are not known a-priori. For example, speech recognition and machine translation are sequential problems. Likewise, question answering can also be seen as mapping a sequence of words representing the question to a sequence of words representing the answer. It is therefore clear that a domain-independent method that learns to map sequences to sequences would be useful.

然而,尽管 DNN 灵活而强大,它只能被应用于那些输入与目标都能合理地用固定维度向量表示的问题。这是一个相当大的局限,因为许多重要的问题,最自然的表示形式是长度事先未知的序列。例如,语音识别和机器翻译都是序列问题。同样地,问答系统也可以被看作是表示问题的词序列映射为表示答案的词序列。因此,显然,一种与领域无关的、能学习“序列到序列”映射的方法将是非常有用的。

Sequences pose a challenge for DNNs because they require that the dimensionality of the inputs and outputs is known and fixed. In this paper, we show that a straightforward application of the Long Short-Term Memory (LSTM) architecture can solve general sequence to sequence problems. The idea is to use one LSTM to read the input sequence, one timestep at a time, to obtain a large fixed-dimensional vector representation, and then to use another LSTM to extract the output sequence from that vector (fig. 1). The second LSTM is essentially a recurrent neural network language model except that it is conditioned on the input sequence. The LSTM’s ability to successfully learn on data with long range temporal dependencies makes it a natural choice for this application due to the considerable time lag between the inputs and their corresponding outputs.

序列之所以对 DNN 构成挑战,是因为它要求输入和输出的维度必须事先已知且固定。本文表明,对长短期记忆网络 (LSTM) 架构进行一种直接的、自然的运用,就可以解决一般的序列到序列问题。其核心思想是:用一个 LSTM 一次一个时间步地读取输入序列,从而得到一个固定维度的“大向量”表示;然后再用另一个 LSTM 从这个向量中“提取”出输出序列(如图 1 所示)。第二个 LSTM 本质上是一个循环神经网络语言模型,只是以输入序列作为条件。LSTM 具有在具有长时间跨度依赖的数据上成功学习的能力,这种能力使其成为本应用的理想选择,因为输入与对应输出之间存在着相当大的时间延迟。

There have been a number of related attempts to address the general sequence to sequence learning problem with neural networks. Our approach is closely related to Kalchbrenner and Blunsom (2013) who were the first to map the entire input sentence to vector, and is related to Cho et al. (2014) although the latter was used only for rescoring hypotheses produced by a phrase-based system. Graves (2013) introduced a novel differentiable attention mechanism that allows neural networks to focus on different parts of their input, and an elegant variant of this idea was successfully applied to machine translation by Bahdanau et al. (2014). The Connectionist Sequence Classification is another popular technique for mapping sequences to sequences with neural networks, but it assumes a monotonic alignment between the inputs and the outputs.

过去已经有一些相关工作尝试用神经网络解决一般的序列到序列学习问题。本文的方法与 Kalchbrenner 与 Blunsom (2013) 的工作紧密相关——他们是第一个将整句输入映射为一个向量的人;也与 Cho 等人 (2014) 的工作相关,不过后者只是被用来对基于短语的统计机器翻译系统所产生的候选翻译进行重排序。Graves (2013) 提出了一种新颖的可微分注意力 (attention) 机制,使得神经网络能够聚焦于输入的不同部分;这一思想的一个优雅变体被 Bahdanau 等人 (2014) 成功应用于机器翻译。Connectionist Sequence Classification 是另一种用神经网络进行序列到序列映射的常用技术,但它要求输入与输出之间是单调对齐的。

Figure 1 — Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier. | ■ 1 — ■■■■■■ “ABC”, ■ ■ “WXYZ”; ■■■■■■ (end-of-sentence) ■■■■■■ LSTM ■■■■■■,

The main result of this work is the following. On the WMT'14 English to French translation task, we obtained a BLEU score of 34.81 by directly extracting translations from an ensemble of 5 deep LSTMs (with 384M parameters and 8,000 dimensional state each) using a simple left-to-right beam-search decoder. This is by far the best result achieved by direct translation with large neural networks. For comparison, the BLEU score of an SMT baseline on this dataset is 33.30. The 34.81 BLEU score was achieved by an LSTM with a vocabulary of 80k words, so the score was penalized whenever the reference translation contained a word not covered by these 80k. This result shows that a relatively unoptimized small-vocabulary neural network architecture which has much room for improvement outperforms a phrase-based SMT system.

本文的主要实验结果如下:在 WMT ' 14 英语—法语翻译任务上,通过 5 个深度 LSTM(每个具有 3.84 亿参数和 8000 维隐藏状态)组成的集成模型,使用一个简单的从左到右的束搜索 (beam search) 解码器直接抽取译文,获得了 34.81 的 BLEU 分数。这是迄今为止直接使用大型神经网络进行翻译所取得的最好结果。作为对比,基于短语的统计机器翻译基线系统在同一数据集上的 BLEU 分数为 33.30。该 34.81 的 BLEU 分数是由词表为 80 000 词的 LSTM 取得的,因此每当参考译文中含有这 80 000 词之外的词时,分数就会受到惩罚。这一结果表明,一个相对未经精细调优、词表较小、但仍有很大改进空间的神经网络架构,能够超过基于短语的统计机器翻译系统。

Finally, we used the LSTM to rescore the publicly available 1000-best lists of the SMT baseline on the same task. By doing so, we obtained a BLEU score of 36.5, which improves the baseline by 3.2 BLEU points and is close to the previous best published result on this task (which is 37.0).

最后,我们使用 LSTM 对同一任务上 SMT 基线系统公开的 1000-best 候选列表进行重排序。由此得到的 BLEU 分数为 36.5,相对基线提升了 3.2 个 BLEU 点,非常接近该任务当时最佳的公开结果(37.0)。

Surprisingly, the LSTM did not suffer on very long sentences, despite the recent experience of other researchers with related architectures. We were able to do well on long sentences because we reversed the order of words in the source sentence but not the target sentences in the training and test set. By doing so, we introduced many short term dependencies that made the optimization problem much simpler (see sec. 2 and sec. 3.3). As a result, SGD could learn LSTMs that had no trouble with long sentences. The simple trick of reversing the words in the source sentence is one of the key technical contributions of this work.

令人惊讶的是,LSTM 在很长的句子上并没有出现性能下降,而其他研究者近期在类似架构上的经验并非如此。我们之所以能在长句上表现良好,是因为我们在训练集和测试集中反转了源语句中的词序,但不反转目标句的词序。这样做在数据中引入了大量短期依赖,显著降低了优化问题的复杂度(详见第 2 节和第 3.3 节)。其结果是,SGD 可以成功训练出能从容应对长句的 LSTM。在源语句中反转词序这一看似简单的小技巧,是本文的关键技术贡献之一。

A useful property of the LSTM is that it learns to map an input sentence of variable length into a fixed-dimensional vector representation. Given that translations tend to be paraphrases of the source sentences, the translation objective encourages the LSTM to find sentence representations that capture their meaning, as sentences with similar meanings are close to each other while different sentences meanings will be far. A qualitative evaluation supports this claim, showing that our model is aware of word order and is fairly invariant to the active and passive voice.

LSTM 的一个有用性质是:它能学会将变长的输入句映射为一个固定维度的向量表示。由于翻译往往是对源句的转述,翻译目标会促使 LSTM 找到能捕捉句意的句子表示——含义相近的句子彼此距离较近,而含义不同的句子彼此距离较远。定性实验支持了这一说法,显示我们的模型能感知词序,并且对主动 / 被动语态的替换相对稳健。

2. The Model

模型

The Recurrent Neural Network (RNN) is a natural generalization of feedforward neural networks to sequences. Given a sequence of inputs ($x_1, \dots, x_T$), a standard RNN computes a sequence of outputs ($y_1, \dots, y_T$) by iterating the following equation: $h_t = \text{sigm}(W^{hx} x_t + W^{hh} h_{t-1})$; $y_t = W^{yh} h_t$. The RNN can easily map sequences to sequences whenever the alignment between the inputs and the outputs is known ahead of time. However, it is not clear how to apply an RNN to problems whose input and the output sequences have different lengths with complicated and non-monotonic relationships.

循环神经网络 (RNN) 是前馈神经网络向序列的自然推广。给定一个输入序列 $(x_1, \dots, x_T)$, 标准 RNN 通过反复迭代以下方程来计算输出序列 $(y_1, \dots, y_T)$: $h_t = \text{sigm}(W^{hx}x_t + W^{hh}h_{t-1})$; $y_t = W^{yh}h_t$ 。只要输入与输出之间的对齐关系事先已知, RNN 就能容易地完成序列到序列的映射。然而, 对于输入序列和输出序列长度不同、且关系复杂且非单调的问题, RNN 应当如何使用并不明朗。

The simplest strategy for general sequence learning is to map the input sequence to a fixed-sized vector using one RNN, and then to map the vector to the target sequence with another RNN (this approach has also been taken by Cho et al. (2014)). While it could work in principle since the RNN is provided with all the relevant information, it would be difficult to train the RNNs due to the resulting long term dependencies (fig. 1). However, the Long Short-Term Memory (LSTM) is known to learn problems with long range temporal dependencies, so an LSTM may succeed in this setting.

对于一般性的序列学习, 最简单的策略是: 用一个 RNN 将输入序列映射为一个固定大小的向量, 再用另一个 RNN 将该向量映射为目标序列 (Cho 等人 (2014) 也采用了这一方案)。原则上这种做法应该可行, 因为 RNN 在每一步都能获得所有相关信息; 然而, 由于输入与输出之间存在长时间跨度的依赖关系 (如图 1 所示), 这种模型的训练会非常困难。但长短期记忆网络 (LSTM)

已被证明能够在具有长程时间依赖性的问题上成功学习, 因此在该场景下 LSTM 应当能够成功。

The goal of the LSTM is to estimate the conditional probability $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ where $(x_1, \dots, x_T)$ is an input sequence and $y_1, \dots, y_{T'}$ is its corresponding output sequence whose length T' may differ from T . The LSTM computes this conditional probability by first obtaining the fixed-dimensional representation v of the input sequence $(x_1, \dots, x_T)$ given by the last hidden state of the LSTM, and then computing the probability of $y_1, \dots, y_{T'}$ with a standard LSTM-LM formulation whose initial hidden state is set to the representation v of $x_1, \dots, x_T$: $p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1})$.

LSTM 的目标是估计条件概率 $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$, 其中 $(x_1, \dots, x_T)$ 是输入序列, $y_1, \dots, y_{T'}$ 是对应的输出序列, 其长度 T' 可能与 T 不相等。LSTM 通过以下方式计算这个条件概率: 首先, 输入序列 $(x_1, \dots, x_T)$ 由 LSTM 的最后一个隐藏状态给出其固定维度的向量表示 v ; 然后, 以 v 作为初始隐藏状态, 使用标准的 LSTM 语言模型公式来计算 $y_1, \dots, y_{T'}$ 的概率: $p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1})$ 。

In this equation, each $p(y_t | v, y_1, \dots, y_{t-1})$ distribution is represented with a softmax over all the words in the vocabulary. We use the LSTM formulation from Graves (2013). Note that we require that each sentence ends with a special end-of-sentence symbol “”, which enables the model to define a distribution over sequences of all possible lengths. The overall scheme is outlined in fig. 1, where the shown LSTM computes the representation of “A”, “B”, “C”, “” and then uses this representation to compute the probability of “W”, “X”, “Y”, “Z”, “”.

在上述公式中, 每个分布 $p(y_t | v, y_1, \dots, y_{t-1})$ 用词表上所有词上的 softmax 来表示。我们采用 Graves (2013) 所提出的 LSTM 公式。需要注意的是, 我们要求每个句子都以一个特殊的句末符号 “” 结尾, 这样模型就可以对所有可能长度的序列定义分布。整个方案的示意图如图 1 所示: 图中 LSTM 先计算出 “A”、“B”、“C”、“” 的表示, 然后再用该表示计算 “W”、“X”、“Y”、“Z”、“” 的概率。

Our actual models differ from the above description in three important ways. First, we used two different LSTMs: one for the input sequence and another for the output sequence, because doing so increases the number of model parameters at negligible computational cost and makes it natural to train the LSTM on multiple language pairs simultaneously. Second, we found that deep LSTMs significantly outperformed shallow LSTMs, so we chose an LSTM with four layers. Third, we found it extremely valuable to reverse the order of the words of the input sentence. So for example, instead of mapping the sentence a, b, c to the sentence α, β, γ , the LSTM is asked to map c, b, a to α, β, γ , where α, β, γ is the translation of a, b, c. This way, a is in close proximity to α , b is fairly close to β , and so on, a fact that makes it easy for SGD to “establish communication” between the input and the output. We found this simple data transformation to greatly improve the performance of the LSTM.

我们实际的模型与上述描述在三个重要方面有所不同。第一, 我们使用了两个不同的 LSTM:

一个用于输入序列, 另一个用于输出序列。这样做在几乎不增加计算开销的前提下增加

了模型参数量, 并且使同时在多种语言对上训练 LSTM 变得自然。第二, 我们发现深度 LSTM 显著优于浅层 LSTM, 因此选用了 4 层的 LSTM。第三, 我们发现将源句的词序反转非常有价值。举例来说, 不是将句子 a, b, c 映射为 α, β, γ , 而是要求 LSTM 将 c, b, a 映射到 α, β, γ (其中 α, β, γ 是 a, b, c 的翻译)。这样, a 就与 α 在位置上靠得很近, b 也与 β 相对靠近, 以此类推。这一事实让 SGD

很容易在输入与输出之间 “建立通信”。我们发现, 这种简单的数据变换可以极大地提升 LSTM 的性能。

3. Experiments

实验

We applied our method to the WMT'14 English to French MT task in two ways. We used it to directly translate the input sentence without using a reference SMT system and we used it to rescore the n-best lists of an SMT baseline. We report the accuracy of these translation methods, present sample translations, and visualize the resulting sentence representation.

我们将本文的方法以两种方式应用于 WMT'14 英—法机器翻译任务。一是直接对输入句进行翻译,不使用任何参考 SMT 系统;二是对一个 SMT 基线系统所产生的 n-best 候选列表进行重排序。我们报告这些翻译方法的准确度,给出若干翻译样例,并对所学到的句子表示进行可视化。

3.1 Dataset Details

数据集细节

We used the WMT'14 English to French dataset. We trained our models on a subset of 12M sentences consisting of 348M French words and 304M English words, which is a clean “selected” subset from Schwenk (2014). We chose this translation task and this specific training set subset because of the public availability of a tokenized training and test set together with 1000-best lists from the baseline SMT system.

我们使用 WMT'14 英—法数据集。我们在由 1200 万句组成的子集(包含 3.48 亿法语词和 3.04 亿英语词)上训练模型,这是 Schwenk (2014) 提供的一个干净的“精选”子集。选择这个翻译任务和这个特定训练子集的原因是:它公开提供了分词后的训练集与测试集,并且能够获得 SMT 基线系统的 1000-best 候选列表。

As typical neural language models rely on a vector representation for each word, we used a fixed vocabulary for both languages. We used 160,000 of the most frequent words for the source language and 80,000 of the most frequent words for the target language. Every out-of-vocabulary word was replaced with a special “UNK” token.

由于典型的神经语言模型依赖每个词的向量表示,我们对两种语言都使用了固定词表。源语言使用 160 000 个最高频词,目标语言使用 80 000 个最高频词。每个词表外的词都被替换为特殊标记“UNK”。

3.2 Decoding and Rescoring

解码与重排序

The core of our experiments involved training a large deep LSTM on many sentence pairs. We trained it by maximizing the log probability of a correct translation T given the source sentence S , so the training objective is $(1/|S|) \sum_{(T,S) \in S} \log p(T|S)$, where S is the training set. Once training is complete, we produce translations by finding the most likely translation according to the LSTM: $T^* = \operatorname{argmax}_T p(T|S)$. We search for the most likely translation using a simple left-to-right beam search decoder which maintains a small number B of partial hypotheses, where a partial hypothesis is a prefix of some translation. At each timestep we extend each partial hypothesis in the beam with every possible word in the vocabulary. This greatly increases the number of the hypotheses so we discard all but the B most likely hypotheses according to the model's log probability. As soon as the “” symbol is appended to a hypothesis, it is removed from the beam and is added to the set of complete hypotheses. While this decoder is approximate, it is simple to implement. Interestingly, our system performs well even with a beam size of 1, and a beam of size 2 provides most of the benefits of beam search.

我们实验的核心是在大量句对上训练一个大型的深度 LSTM。我们通过对数似然最大化进行训练:给定源句 S , 最大化正确译文 T 的对数概率, 目标函数为 $(1/|S|) \sum_{(T,S) \in S} \log p(T|S)$, 其中 S 是训练集。训练完成后, 通过寻找 LSTM 下的最可能译文来生成译文: $T^* = \operatorname{argmax}_T p(T|S)$ 。我们用一个简单的从左到右的束搜索解码器来寻找最可能的译文, 该解码器维护少量 B 个“部分假设”, 其中部分假设是某个译文的前缀。在每个时间步, 我们将束中的每个部分假设用词表中的每一个可能词进行扩展, 这会导致候选数大幅增加, 于是我们按模型对数概率只保留 B 个最可能的假设。一旦某个假设后接了 “”, 就将它从束中移出, 并加入到完整假设集合中。虽然这个解码器是近似的, 但实现起来非常简单。有趣的是, 即使束大小为 1, 我们的系统也能表现良好; 而束大小为 2 就能获得束搜索的绝大部分收益。

We also used the LSTM to rescore the 1000-best lists produced by the baseline system. To rescore an n-best list, we computed the log probability of every hypothesis with our LSTM and took an even average with their score and the LSTM's score.

我们还使用 LSTM 对基线系统产生的 1000-best 候选列表进行重排序。对 n-best 列表进行重排序时, 我们用 LSTM 计算每一条假设的对数概率, 并将其分数与 LSTM 分数取算术平均。

3.3 Reversing the Source Sentences

反转源句词序

While the LSTM is capable of solving problems with long term dependencies, we discovered that the LSTM learns much better when the source sentences are reversed (the target sentences are not reversed). By doing so, the LSTM's test perplexity dropped from 5.8 to 4.7, and the test BLEU scores of its decoded translations increased from 25.9 to 30.6.

尽管 LSTM 具备解决长程依赖问题的能力,我们发现,当源语句词序被反转时(目标语句词序 保持不变),LSTM 学得更好。这样处理后,LSTM 的测试困惑度从 5.8 下降到 4.7,其解码 所得译文的测试 BLEU 分数从 25.9 提升到 30.6。

While we do not have a complete explanation to this phenomenon, we believe that it is caused by the introduction of many short term dependencies to the dataset. Normally, when we concatenate a source sentence with a target sentence, each word in the source sentence is far from its corresponding word in the target sentence. As a result, the problem has a large “minimal time lag”. By reversing the words in the source sentence, the average distance between corresponding words in the source and target language is unchanged. However, the first few words in the source language are now very close to the first few words in the target language, so the problem's minimal time lag is greatly reduced. Thus, backpropagation has an easier time “establishing communication” between the source sentence and the target sentence, which in turn results in substantially improved overall performance.

虽然我们尚无完整的解释,但我们相信这一现象是因为在数据集中引入了大量“短期依赖”。通常情况下,当我们把源句和目标句拼接起来时,源句中的每个词都离它在目标句中对应的词很远,因此问题具有较大的“最小时间延迟”。反转源句中的词序后,源语言与目标语言中对应词之间的平均距离并未改变。然而,源语言开头的若干词现在与目标语言开头的若干词非常接近,于是问题的“最小时间延迟”被显著缩短。这样,反向传播就更容易在源句与目标句之间“建立通信”,从而显著提升整体性能。

Initially, we believed that reversing the input sentences would only lead to more confident predictions in the early parts of the target sentence and to less confident predictions in the later parts. However, LSTMs trained on reversed source sentences did much better on long sentences than LSTMs trained on the raw source sentences (see sec. 3.7), which suggests that reversing the input sentences results in LSTMs with better memory utilization.

最初我们以为,反转输入句只会让目标句前部分的预测更自信、目标句后部分的预测更不自信。然而,在反转源句上训练的 LSTM 在长句上反而比在原始源句上训练的 LSTM 表现更好(见第 3.7 节)。这表明,反转源句能够获得对内存利用更高效的 LSTM。

3.4 Training Details

训练细节

We found that the LSTM models are fairly easy to train. We used deep LSTMs with 4 layers, with 1000 cells at each layer and 1000 dimensional word embeddings, with an input vocabulary of 160,000 and an output vocabulary of 80,000. Thus the deep LSTM uses 8000 real numbers to represent a sentence. We found deep LSTMs to significantly outperform shallow LSTMs, where each additional layer reduced perplexity by nearly 10%, possibly due to their much larger hidden state. We used a naive softmax over 80,000 words at each output. The resulting LSTM has 384M parameters of which 64M are pure recurrent connections (32M for the “encoder” LSTM and 32M for the “decoder” LSTM). The complete training details are given below:

我们发现 LSTM 模型相当容易训练。我们使用 4 层的深度 LSTM,每层 1000 个单元,词嵌入维度为 1000;源端词表为 160 000,目标端词表为 80 000。因此,深度 LSTM 用 8000 个实数表示一个句子。我们发现深度 LSTM 显著优于浅层 LSTM,每增加一层,困惑度几乎下降 10%,这可能得益于它们更大的隐藏状态。在每个输出位置我们使用朴素的、词表大小为 80 000 的 softmax。最终得到的 LSTM 共有 3.84 亿参数,其中 6400 万为纯粹的循环连接(“编码器” LSTM 占 3200 万,“解码器” LSTM 占 3200 万)。完整训练细节如下:

- We initialized all of the LSTM's parameters with the uniform distribution between -0.08 and 0.08 .
- We used stochastic gradient descent without momentum, with a fixed learning rate of 0.7 . After 5 epochs, we begun halving the learning rate every half epoch. We trained our models for a total of 7.5 epochs.
- We used batches of 128 sequences for the gradient and divided it the size of the batch (namely, 128).
- Although LSTMs tend to not suffer from the vanishing gradient problem, they can have exploding gradients. Thus we enforced a hard constraint on the norm of the gradient by scaling it when its norm exceeded a threshold. For each training batch, we compute $s = \frac{g}{\|g\|}$, where g is the gradient divided by 128. If $s >$

5, we set $g = 5g/s$. • Different sentences have different lengths. Most sentences are short (e.g., length 20–30) but some sentences are long (e.g., length > 100), so a minibatch of 128 randomly chosen training sentences will have many short sentences and few long sentences, and as a result, much of the computation in the minibatch is wasted. To address this problem, we made sure that all sentences in a minibatch are roughly of the same length, yielding a 2× speedup.

- 我们用 $(-0.08, 0.08)$ 区间上的均匀分布初始化 LSTM 的所有参数。

我们使用不带动量的随机梯度下降(SGD),固定学习率为 0.7。训练 5 个 epoch 后,我们开始每半个 epoch 将学习率减半;模型一共训练 7.5 个 epoch。

- 梯度按大小为 128 的序列批量计算,并除以批大小(即 128)。
- 虽然 LSTM 通常不存在梯度消失的问题,但可能会出现梯度爆炸。因此我们对梯度的范数施加了一个硬约束:当梯度范数超过阈值时对其做缩放。具体做法是:对每个训练批,计算 $s = \|g\|_2$, 其中 g 为除以 128 后的梯度。若 $s > 5$, 则令 $g = 5g/s$ 。
- 不同句子长度差别很大。大部分句子较短(例如 20–30 个词),但有些句子很长(例如长度 > 100)。因此,从训练集中随机抽取 128 句构成的 minibatch 中短句居多、长句稀少,造成 minibatch 中大量计算被浪费。为解决这一问题,我们使每个 minibatch 内的句子长度大致接近,从而获得约 2 倍的加速。

3.5 Parallelization

并行化

A C++ implementation of deep LSTM with the configuration from the previous section on a single GPU processes a speed of approximately 1,700 words per second. This was too slow for our purposes, so we parallelized our model using an 8-GPU machine. Each layer of the LSTM was executed on a different GPU and communicated its activations to the next GPU / layer as soon as they were computed. Our models have 4 layers of LSTMs, each of which resides on a separate GPU. The remaining 4 GPUs were used to parallelize the softmax, so each GPU was responsible for multiplying by a 1000×20000 matrix. The resulting implementation achieved a speed of 6,300 (both English and French) words per second with a minibatch size of 128. Training took about ten days with this implementation.

对于上一节配置的深度 LSTM,基于 C++ 的实现在单 GPU 上每秒大约能处理 1 700 个词,这对我们来说太慢了。因此,我们将模型并行化到一台 8 卡 GPU 的机器上。LSTM 的每一层运行在不同的 GPU 上,一旦计算出激活值,立即将其发送到对应下一层 / 下一 GPU。我们的模型有 4 层 LSTM,每层单独占用一块 GPU;其余 4 块 GPU 用来并行 softmax,即每块 GPU 负责与一个 $1000 \times 20\,000$ 维矩阵相乘。最终实现的处理速度为 6 300 个(英 / 法)词 / 秒,minibatch 大小为 128。该实现训练大约需要 10 天。

3.6 Experimental Results

实验结果

We used the cased BLEU score to evaluate the quality of our translations. We computed our BLEU scores using multi-bleu.pl on the tokenized predictions and ground truth. This way of evaluating the BLEU score is consistent with Cho et al. (2014) and Bahdanau et al. (2014), and reproduces the 33.3 score of Schwenk (2014). However, if we evaluate the best WMT'14 system (whose predictions can be downloaded from statmt.org) in this manner, we get 37.0, which is greater than the 35.8 reported by statmt.org.

我们使用区分大小写的 BLEU 分数来评估译文质量。使用 multi-bleu.pl 在分词后的预测结果与参考译文上计算 BLEU 分数。这种评估方式与 Cho 等人(2014)和 Bahdanau 等人 (2014)一致,并复现了 Schwenk (2014) 的 33.3 这一分数。然而,如果用同样方式评估 WMT ' 14 最好的系统(其预测可以从 statmt.org 下载),我们会得到 37.0,高于 statmt.org 报告的 35.8。

The results are presented in Table 1 and Table 2. Our best results are obtained with an ensemble of LSTMs that differ in their random initializations and in the random order of minibatches. While the decoded translations of the LSTM ensemble do not outperform the best WMT'14 system, it is the first time that a pure neural translation system outperforms a phrase-based SMT baseline on a large scale MT task by a sizeable margin, despite its inability to handle out-of-vocabulary words. The LSTM is within 0.5 BLEU points of the best WMT'14 result if it is used to rescore the 1000-best list of the baseline system.

实验结果如表 1 和表 2 所示。我们最好的结果来自一个 LSTM 集成模型,其中的 LSTM 在随机初始化和 minibatch 随机顺序上互不相同。虽然 LSTM 集成模型的解码译文仍未超过 WMT ' 14 最好的系统,但它首次在如此大规模的机器翻译任务上,以明显的优势超过了一个基于短语的 SMT 基线系统——尽管它还无法处理词表外的词。当 LSTM 被用于对基线系统的 1000-best 候选列表进行重排序时,其在 BLEU 上仅与 WMT ' 14 最佳结果相差 0.5 点。

结论

在本文中,我们证明:一个大型的、词表有限的深度 LSTM,几乎不依赖任何问题结构上的假设,就能在一个大规模的机器翻译任务上超过一个词汇不受限制的标准 SMT 系统。我们这种简单的基于 LSTM 的方法在机器翻译上的成功表明:只要拥有足够的训练数据,它也应该能在许多其他序列学习问题上表现良好。

反转源语句词序所带来的提升幅度令我们感到惊讶。由此我们得出结论:寻找一种具有尽可能多短期依赖的问题编码方式非常重要,因为短期依赖会使学习问题变得简单得多。具体而言,虽然我们无法在未反转的翻译问题上成功训练一个标准 RNN(如图 1 所示),但我们认为:只要将源语句反转,标准 RNN 也应当能够被轻易地训练起来(尽管我们没有进行实验验证)。

LSTM 仍能正确翻译很长的句子,这一点也令我们感到惊讶。我们原本以为 LSTM 由于其有限的记忆容量,会在长句上失败;其他研究者在与我们类似的模型上也报告过在长句上表现不佳。然而,在反转源句的数据集上训练的 LSTM,翻译长句却毫无困难。

最重要的是,我们证明了一个简单、直接、且相对未经精细调优的方法可以超过 SMT 系统,因此后续的工作有望带来更高的翻译准确度。这些结果表明,本文的方法很可能在其他具有挑战性的序列到序列问题上同样表现良好。

致谢

我们感谢 Samy Bengio、Jeff Dean、Matthieu Devin、Geoffrey Hinton、Nal Kalchbrenner、Thang Luong、Wolfgang Macherey、Rajat Monga、Vincent Vanhoucke、Peng Xu、Wojciech Zaremba 以及 Google Brain 团队所提供的宝贵意见与讨论。

[illegible]

Attention Is All You Need

注意力机制就是你需要的一切

Ashish Vaswani · Noam Shazeer · Niki Parmar · Jakob Uszkoreit · Llion Jones · Aidan N. Gomez · Lukasz Kaiser · Illia Polosukhin
Google Brain / Google Research

阿希什·瓦斯瓦尼 · 诺姆·沙泽尔 · 尼基·帕尔马 · 雅各布·乌什科赖特 · 利昂·琼斯 · 艾丹·N·戈麦斯 · 乌卡什·凯泽 · 伊利亚·波洛苏欣(均来自 Google)

Advances in Neural Information Processing Systems 30 (NIPS 2017), pp. 5998–6008 | arXiv:1706.03762 [cs.CL]

Abstract

摘要

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

当前主流的序列转换(sequence transduction)模型都基于复杂的循环神经网络或卷积神经网络,这些网络通常包含一个编码器和一个解码器。表现最好的模型还会通过注意力机制将编码器与解码器连接起来。我们提出了一种全新的、简洁的网络架构——Transformer,它完全基于注意力机制,完全摒弃了循环与卷积。在两个机器翻译任务上的实验表明,这些模型在质量上更优,同时具有更高的并行度,训练所需时间也显著减少。在 WMT 2014 英—德翻译任务上,我们的模型取得了 28.4 的 BLEU 分数,比当时已有的最佳结果(包括集成模型)还高出 2 BLEU 以上。在 WMT 2014 英—法翻译任务上,我们的模型在 8 块 GPU 上仅训练了 3.5 天,便以单一模型取得了 41.8 的 BLEU 分数,刷新了该任务的最佳成绩,而训练成本只是文献中最优模型的一小部分。我们还通过将 Transformer 成功应用于英文成分句法分析(无论训练数据多寡)展示了它能够良好地泛化到其他任务。

1. Introduction

引言

Recurrent neural networks, long short-term memory and gated recurrent neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures.

循环神经网络,尤其是长短期记忆网络 (LSTM) 和门控循环单元 (GRU),已经成为序列建模与序列转换任务(如语言建模和机器翻译)中公认的最先进方法。此后,众多研究继续推动着循环语言模型和编码器—解码器架构的边界。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks and conditional computation, while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

循环模型通常沿输入与输出序列的符号位置来分解计算。将这些位置与计算时间步对齐后,模型会根据前一隐藏状态 h_{t-1} 与位置 t 处的输入,生成当前隐藏状态 h_t 。这种内在的顺序性使得同一训练样本内部难以并行化,而随着序列变长,这会成为关键瓶颈——内存约束限制了跨样本的批大小。近期工作通过分解技巧与条件计算在计算效率上取得了显著改进,后者还同时提升了模型性能。然而,顺序计算这一根本性约束依然存在。

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences. In all but a few cases, however, such attention mechanisms are used in conjunction with a recurrent network.

注意力机制已经成为许多引人注目的序列建模与转换模型的核心组成,使模型能够刻画依赖关系而不必在意其在输入或输出序列中的距离。然而,除少数情况外,这类注意力机制仍与循环网络结合使用。

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

本文提出了一种名为 Transformer 的模型架构,它完全摒弃了循环结构,而是仅依靠注意力机制在输入与输出之间建立全局依赖。Transformer 显著提高了并行度,只需在 8 块 P100 GPU 上训练约 12 小时,即可在翻译质量上达到新的最优水平。

2. Background

背景

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU, ByteNet and ConvS2S, all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in Section 3.2.

减少顺序计算这一目标也是 Extended Neural GPU、ByteNet 和 ConvS2S 这些工作的共同出发点:它们都使用卷积神经网络作为基本构建块,为所有输入和输出位置并行地计算隐藏表示。在这些模型中,关联任意两个输入或输出位置所需的运算量随位置之间的距离增长而增长——ConvS2S 中线性增长,ByteNet 中对数增长。这使得学习远距离依赖变得更加困难。而在 Transformer 中,这一开销被降低为常数;代价是因为对注意力加权位置进行了平均,有效分辨率有所下降,我们将在 3.2 节中通过多头注意力机制抵消这一影响。

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations.

自注意力(self-attention,有时也称为 intra-attention)是一种关联同一序列中不同位置的注意力机制,用于计算该序列的表示。自注意力已在多种任务中取得成功,包括阅读理解、抽象摘要、文本蕴含以及与具体任务无关的句子表示学习。

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks.

端到端记忆网络(end-to-end memory networks)基于循环注意力机制而非按序列对齐的循环结构,在简单的语言问答和语言建模任务上表现良好。

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as neural GPUs, ByteNet and ConvS2S.

据我们所知,Transformer 是首个完全依赖自注意力来计算输入与输出表示的转换模型,它不使用任何按序列对齐的 RNN 或卷积。在接下来的章节中,我们将描述 Transformer 架构、阐述自注意力的动机,并讨论它相对于 Neural GPU、ByteNet 和 ConvS2S 等模型的优势。

3. Model Architecture

模型架构

Most competitive neural sequence transduction models have an encoder-decoder structure. Here, the encoder maps an input sequence of symbol representations ($x_1, \dots, x_n$) to a sequence of continuous representations $z = (z_1, \dots, z_n)$. Given z , the decoder then generates an output sequence ($y_1, \dots, y_m$) of symbols one element at a time. At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next.

大多数有竞争力的神经序列转换模型都采用编码器—解码器结构。其中,编码器将一个由符号表示组成的输入序列 ($x_1, \dots, x_n$) 映射为连续表示序列 $z = (z_1, \dots, z_n)$; 给定 z , 解码器再一个符号一个符号地生成输出序列 ($y_1, \dots, y_m$)。在每一步,模型以自回归方式工作,将先前已生成的符号作为额外输入来生成下一个符号。

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

Transformer 沿用这种总体架构,在编码器和解码器中都使用堆叠的自注意力层和逐位 (point-wise) 全连接层,分别如图 1 的左半部分和右半部分所示。

3.1 Encoder and Decoder Stacks

编码器与解码器堆叠

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection around each of the two sub-layers, followed by layer normalization. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

编码器: 编码器由 $N = 6$ 个完全相同的层堆叠而成。每一层包含两个子层。第一个子层是多头自注意力机制,第二个子层是简单的逐位置(position-wise)全连接前馈网络。我们在两个子层周围都使用了残差连接(residual connection),再进行层归一化(layer normalization)。即每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$, 其中 $\text{Sublayer}(x)$ 为该子层自身实现的变换。为便于这些残差连接,模型中所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

解码器: 解码器同样由 $N = 6$ 个完全相同的层堆叠而成。除了编码器层中的两个子层之外,解码器还插入了一个第三子层,该子层对编码器栈的输出执行多头注意力。与编码器类似,我们在每个子层周围使用残差连接,并进行层归一化。我们还修改变码器栈中的自注意力子层,以防止位置关注到后续位置。这种掩码(masking)机制与输出嵌入偏移一个位置相结合,确保对位置 i 的预测只能依赖于位置小于 i 的已知输出。

3.2 Attention

注意力

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

注意力函数可以描述为:将一个查询(query)和一组键—值(key-value)对映射为一个输出,其中查询、键、值和输出都是向量。输出是各值向量的加权和,每个值向量所对应的权重由查询与相应键的兼容性函数(compatibility function)计算得到。

3.2.1 Scaled Dot-Product Attention

缩放点积注意力

We call our particular attention “Scaled Dot-Product Attention” (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

我们将本文所采用的注意力机制称为缩放点积注意力(Scaled Dot-Product Attention,图 2)。输入由维度为 d_k 的查询和键以及维度为 d_v 的值组成。我们计算查询与所有键的点积,将每个点积除以 $\sqrt{d_k}$,然后对其应用 softmax,得到对各值的权重。

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as the equation above.

实际实现时,我们会在一组查询上同时计算注意力函数,将这些查询打包成矩阵 Q ;键和值也分别打包成矩阵 K 和 V 。我们按上式计算输出矩阵。

The two most commonly used attention functions are additive attention, and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $1/\sqrt{d_k}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

两种最常用的注意力函数分别是加性注意力(additive attention)和点积(乘性)注意力。

点积注意力与本文算法完全一致,只是多了 $1/\sqrt{d_k}$ 这一缩放因子。加性注意力使用一个单隐藏层的前馈网络来计算兼容性函数。两者在理论复杂度上相近,但实践中点积注意力速度更快、空间效率更高,因为它可以通过高度优化的矩阵乘法代码来实现。

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k . We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by $1/\sqrt{d_k}$.

当 d_k 较小时,两种机制表现相近;但 d_k 较大时,加性注意力优于未做缩放的点积注意力。我们推测:对于较大的 d_k ,点积结果的数值会随之变大,从而将 softmax 推入其梯度极小的饱和区间。为抵消这一效应,我们将点积按 $1/\sqrt{d_k}$ 进行缩放。

3.2.2 Multi-Head Attention

多头注意力

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

我们发现,与使用 d_{model} 维的键、值和查询做一次注意力计算相比,将查询、键和值分别通过 h 个不同的、学习到的线性投影映射到 d_k 、 d_k 和 d_v 维,然后在各自的投影版本上并行执行注意力函数,会更有帮助。每路输出为 d_v 维向量,这些向量被拼接起来再经过一次线性投影,得到最终输出,如图 2 所示。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O; \text{head}_i = \text{Attention}(Q W^Q_i, K W^K_i, V W^V_i)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O; \text{head}_i = \text{Attention}(Q W^Q_i, K W^K_i, V W^V_i)$$

Where the projections are parameter matrices $W^Q_i \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^K_i \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^V_i \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$.

其中投影矩阵为参数矩阵 $W^Q_i \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W^K_i \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W^V_i \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$ 。

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

多头注意力使模型能够同时关注来自不同表示子空间、不同位置的信息。而使用单头注意力时,平均操作会抑制这种能力。

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}} / h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

本文中我们使用 $h = 8$ 个并行的注意力层(即头)。每一头的维度为 $d_k = d_v = d_{\text{model}} / h = 64$ 。由于每个头的维度降低,多头注意力的总计算开销与单头全维度注意力相近。

3.2.3 Applications of Attention in our Model

注意力在本文模型中的应用

The Transformer uses multi-head attention in three different ways:

Transformer 在三种不同场景下使用多头注意力:

- In “encoder-decoder attention” layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models.

- 在编码器—解码器注意力层中,查询来自上一层的解码器,而键和值来自编码器的输出。这使得解码器的每个位置都可以关注输入序列中的所有位置。这与典型序列到序列模型中的编码器—解码器注意力机制一致。

- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.

- 编码器包含自注意力层。在自注意力层中,所有键、值和查询都来自同一来源——此处即编码器前一层输出。编码器中的每个位置都可以关注前一层的所有位置。

- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.

- 同样地,解码器中的自注意力层允许解码器的每个位置关注解码器中截至并包括该位置的所有位置。我们需要阻止解码器中的信息向左流动,以保持自回归性质。具体做法是在缩放点积注意力内部,通过将 softmax 输入中所有非法连接对应的值置为 $-\infty$ 来屏蔽掉这些位置。

3.3 Position-wise Feed-Forward Networks

逐位置前馈网络

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

除注意力子层外,编码器和解码器的每一层都包含一个全连接前馈网络,该网络对每个位置分别且相同地应用。它由两层线性变换和中间的一个 ReLU 激活函数组成。

$$FFN(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

$$FFN(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is d_{model}

= 512, and the inner-layer has dimensionality $d_{ff} = 2048$.

虽然不同位置上的线性变换是相同的,但层与层之间使用不同的参数。另一种等价的描述是将其视为两个卷积核大小为 1 的卷积。输入和输出的维度均为 $d_{model} = 512$,内层 维度为 $d_{ff} = 2048$ 。

3.4 Embeddings and Softmax

嵌入与 Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation. In the embedding layers, we multiply those weights by $\sqrt{d_{model}}$.

与其他序列转换模型类似,我们使用学习到的嵌入将输入和输出符号转换为 d_{model} 维的向量。我们也使用常规的学习到的线性变换与 softmax 函数,将解码器的输出转换为下一个标记的预测概率。在本文模型中,两个嵌入层和 softmax 之前的线性变换共享同一权重矩阵。在嵌入层中,我们将这些权重乘以 $\sqrt{d_{model}}$ 。

3.5 Positional Encoding

位置编码

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add “positional encodings” to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed.

由于本文模型既无循环结构也无卷积结构,为使模型能利用序列的顺序信息,我们必须向序列中注入关于符号相对或绝对位置的信息。为此,我们在编码器和解码器栈底部的输入嵌入上叠加了位置编码。位置编码与嵌入具有相同的维度 d_{model} ,因此二者可以直接相加。位置编码的选择既可以是学习得到的,也可以是固定的。

$$\begin{aligned} PE_{\{pos, 2i\}} &= \sin(pos / 10000^{2i / d_{model}}) \\ PE_{\{pos, 2i+1\}} &= \cos(pos / 10000^{2i / d_{model}}) \\ PE_{\{pos, 2i\}} &= \sin(pos / 10000^{2i / d_{model}}) \\ PE_{\{pos, 2i+1\}} &= \cos(pos / 10000^{2i / d_{model}}) \end{aligned}$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , $PE_{\{pos+k\}}$ can be represented as a linear function of $PE_{\{pos\}}$.

其中 pos 表示位置, i 表示维度。也就是说,位置编码的每个维度对应一个正弦波。这些正弦波的波长构成从 2π 到 $10000 \cdot 2\pi$ 的等比数列。我们之所以选择这种函数,是因为我们假设它能让模型更容易学到按相对位置进行关注:对任意固定的偏移 k , $PE_{\{pos+k\}}$ 都可以表示为 $PE_{\{pos\}}$ 的线性函数。

We also experimented with using learned positional embeddings instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

我们也尝试了使用可学习的位置嵌入,发现两种版本的结果几乎相同(见表 3 第 (E) 行)。我们最终选择正弦版本,是因为它可能允许模型在训练时遇到的序列长度之外进行外推。

4. Why Self-Attention

为什么使用自注意力

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations ($x_1, \dots, x_n$) to another sequence of equal length ($z_1, \dots, z_n$), with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder.

一个逐位置前馈层的组合——这正是本文模型所采用的方案。

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

附带的好处是,自注意力有助于产生更具可解释性的模型。我们检查了模型中的注意力分布,并在附录中展示与讨论了相关示例。不仅是各个注意力头明显地学会了完成不同的任务,其中许多还表现出与句子句法结构、语义结构相关的行为。

5. Training

训练

5.1 Training Data and Batching

训练数据与批处理

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding, which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

我们在标准的 WMT 2014 英—德数据集上进行训练,数据集包含约 450 万对句子。句子采用字节对编码(byte-pair encoding),源端与目标端共享一个约 37 000 个标记的词表。对于英—法任务,我们使用了规模更大的 WMT 2014 英—法数据集,包含 3600 万句,并将标记切分为 32 000 个 word-piece 的词表。我们按大致相近的序列长度对句对进行分批;每个训练批包含约 25 000 个源端标记和 25 000 个目标端标记。

5.2 Hardware and Schedule

硬件与训练计划

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models (described on the bottom line of Table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

我们在配备 8 块 NVIDIA P100 GPU 的单台机器上训练模型。对于使用本文中所述超参数的基础模型,每一步训练耗时约 0.4 秒;基础模型总共训练了 100 000 步,合计 12 小时。对于大型模型(对应表 3 最后一行),每步耗时 1.0 秒,训练 300 000 步,合计 3.5 天。

5.3 Optimizer

优化器

We used the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

我们使用 Adam 优化器,其中 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 、 $\epsilon = 10^{-9}$ 。训练过程中,我们按如下公式调整学习率:

$$lr_{rate} = d_model^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$
$$lr_{rate} = d_model^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

This corresponds to increasing the learning rate linearly for the first warmup_steps training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used warmup_steps = 4000.

这等价于:在前 warmup_steps 步中线性增大学习率,之后按步数的反平方根成比例减小。我们设置 warmup_steps = 4000。

5.4 Regularization

正则化

We employ three types of regularization during training:

我们在训练中使用了三种正则化方法:

Residual Dropout: We apply dropout to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{\text{drop}} = 0.1$.

残差 Dropout:我们对每个子层的输出应用 dropout,然后再将其与子层输入相加并归一化。此外,在编码器和解码器栈中,我们对嵌入与位置编码之和也应用 dropout。对于基础模型,我们使用 $P_{\text{drop}} = 0.1$ 的丢弃率。

Label Smoothing: During training, we employed label smoothing of value $\epsilon_{\text{ls}} = 0.1$. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

标签平滑(Label Smoothing):训练中我们采用 $\epsilon_{\text{ls}} = 0.1$ 的标签平滑。虽然这会使模型更加不确定,从而导致困惑度上升,但能提升准确率与 BLEU 分数。

6. Results

实验结果

6.1 Machine Translation

机器翻译

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

在 WMT 2014 英—德翻译任务上,大型 Transformer 模型(表 2 中的 Transformer (big))以 28.4 的 BLEU 分数刷新了当时的最佳成绩,比此前报道的最佳模型(包括集成模型)高出 2.0 BLEU 以上。该模型的配置列于表 3 的最后一行,在 8 块 P100 GPU 上训练了 3.5 天。甚至我们的基础模型也以远低于所有竞品模型的训练成本,超越了此前所有已发表 的模型与集成。

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{\text{drop}} = 0.1$, instead of 0.3.

在 WMT 2014 英—法翻译任务上,我们的大型模型取得了 41.0 的 BLEU 分数,超过了此前所有已发表的单模型成绩,而其训练成本不到此前最佳模型的 1/4。在英—法任务上,Transformer (big) 模型使用的 dropout 率为 $P_{\text{drop}} = 0.1$ (而不是 0.3)。

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible.

对于基础模型,我们将最后 5 个检查点(每 10 分钟保存一次)进行平均,作为最终模型。对于大型模型,我们平均最后 20 个检查点。我们使用束大小为 4、长度惩罚 $\alpha = 0.6$ 的束搜索。这些超参数是在开发集上实验得到的。推理时,我们将最大输出长度设为输入长度 + 50,并在可能时提前终止生成。

Model	EN-DE BLEU	EN-FR BLEU		EN-DE FLOPs	EN-FR FLOPs
ByteNet	23.75	—		—	—
Deep-Att + PosUnk	—	39.2		—	$1.0 \cdot 10^2$ ■
GNMT + RL	24.6	39.92		$2.3 \cdot 10^1$ ■	$1.4 \cdot 10^2$ ■
ConvS2S	25.16	40.46		$9.6 \cdot 10^1$ ■	$1.5 \cdot 10^2$ ■
MoE	26.03	40.56		$2.0 \cdot 10^1$ ■	$1.2 \cdot 10^2$ ■
Deep-Att + PosUnk (Ens.)	—	40.4		—	$8.0 \cdot 10^2$ ■

of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

我们仅在 Section 22 开发集上做了少量实验,用以选择 dropout(注意力 dropout 与残差 dropout)、学习率以及束大小;其余参数保持与英—德翻译基础模型相同。推理时,我们将最大输出长度增加到 $\text{input length} + 300$ 。WSJ-only 与半监督两种设置下,我们都使用束大小 21 与 $\alpha = 0.3$ 。

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar. In contrast to RNN sequence-to-sequence models, the Transformer outperforms the BerkeleyParser even when training only on the WSJ training set of 40K sentences.

表 4 的结果表明:尽管缺乏针对特定任务的调优,本文模型表现惊人地好,在除 Recurrent Neural Network Grammar 之外的所有此前报道的模型中取得了最好的结果。与基于 RNN 的序列到序列模型不同,即便仅使用 40 000 句的 WSJ 训练集,Transformer 也超过了 BerkeleyParser。

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014)	WSJ only, discriminative	88.3
Petrov et al. (2006)	WSJ only, discriminative	90.4
Zhu et al. (2013)	WSJ only, discriminative	90.4
Dyer et al. (2016)	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013)	Semi-supervised	91.3
Huang & Harper (2009)	Semi-supervised	91.3
McClosky et al. (2006)	Semi-supervised	92.1
Vinyals & Kaiser et al. (2014)	Semi-supervised	92.1
Transformer (4 layers)	Semi-supervised	92.7
Luong et al. (2015)	Multi-task	93.0
Dyer et al. (2016)	Generative	93.3

Table 4 — The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ).

[illegible]

7. Conclusion

结论

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

本文中,我们提出了 Transformer——第一个完全基于注意力的序列转换模型。它在编码器—解码器架构中,用多头自注意力替代了最常用的循环层。

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

在翻译任务上,Transformer 的训练速度明显快于基于循环层或卷积层的架构。在 WMT 2014 英—德与英—法两个翻译任务上,我们都达到了新的最佳水平;在前一个任务上,我们的最佳模型甚至超过了所有此前报道的集成模型。

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

我们对基于注意力的模型的未 来充满期待,并计划将其应用于其他任务。我们计划将 Transformer 扩展到文本以外的输入输出模态(如图像、音频和视频)的问题中,并研究局 部化、受限的注意力机制,以高效处理超长输入输出。降低生成过程的顺序性也是我们的 研究方向之一。

The code we used to train and evaluate our models is available at <https://github.com/tensorflow/tensor2tensor>.

我们用于训练与评估模型的代码已开源,地址:<https://github.com/tensorflow/tensor2tensor>。

Acknowledgements

致谢

We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

我们感谢 Nal Kalchbrenner 和 Stephan Gouws 给予的富有成效的评论、纠正与启发。

Note: This is a faithful, full-text translation of the paper as published in NeurIPS 2017. Figures have been referenced by number but not re-rendered; please consult the original paper for figures. All mathematical formulas are reproduced in linear notation (e.g., $QK^T / \sqrt{d_k}$).

Scaling Laws for Neural Language Models

神经语言模型的尺度定律

Jared Kaplan · Sam McCandlish · Tom Henighan · Tom B. Brown · Benjamin Chess · Rewon Child · Scott Gray
· Alec Radford · Jeffrey Wu · Dario Amodei
Johns Hopkins University / OpenAI

贾里德·卡普兰 · 萨姆·麦坎德利斯 · 汤姆·赫尼甘 · 汤姆·B·布朗 · 本杰明·切斯 ·
雷翁·柴尔德 · 斯科特·格雷 · 亚历克·拉德福德 · 杰弗里·吴 · 达里奥·阿莫代i(OpenAI 等)

arXiv:2001.08361 [cs.LG] | Preprint, January 2020

Abstract

摘要

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

我们研究了语言模型性能(交叉熵损失)随尺度变化的实证定律。损失随模型规模、数据集规模以及训练所用的算力呈幂律关系,部分趋势跨越了超过七个数量级。在较宽的范围内,网络宽度或深度等其他架构细节的影响很小。简单的方程即可刻画过拟合对模型/数据规模的依赖,以及训练速度对模型规模的依赖。这些关系使我们能够确定在固定算力预算下的最优分配方式。更大的模型显著更加样本高效,因此,算力最优的训练方式是:用相对适量的数据去训练一个非常大的模型,并在远未收敛时就提前停止训练。

1. Introduction

引言

Language provides a natural domain for the study of artificial intelligence, as the vast majority of reasoning tasks can be efficiently expressed and evaluated in language, and the world's text provides a wealth of data for unsupervised learning via generative modeling. Deep learning has recently seen rapid progress in language modeling, with state of the art models approaching human-level performance on many specific tasks, including the composition of coherent multi-paragraph prompted text samples.

语言是研究人工智能的一个自然领域——绝大多数推理任务都能被高效地用语言表达与评估,而世界上的文本为通过生成式建模进行无监督学习提供了海量数据。近年来,深度学习在语言建模上取得了飞速进展,最先进的方法已在许多特定任务上接近人类水平,包括生成连贯的多段提示文本。

One might expect language modeling performance to depend on model architecture, the size of neural models, the computing power used to train them, and the data available for this training process. In this work we will empirically investigate the dependence of language modeling loss on all of these factors, focusing on the Transformer architecture. The high ceiling and low floor for performance on language tasks allows us to study trends over more than seven orders of magnitude in scale.

人们可能预期,语言建模的性能会依赖于模型架构、神经网络的规模、训练所用的算力,以及可用于训练的数据。在本文中,我们将实证地研究语言建模损失对所有这些因素的依赖,并聚焦于Transformer 架构。语言任务性能的高上限和低下限,使我们能够在超过七个数量级的尺度上研究其趋势。

Throughout we will observe precise power-law scalings for performance as a function of training time, context length, dataset size, model size, and compute budget.

在本文中,我们将观察到:性能作为训练时间、上下文长度、数据集规模、模型规模和算力预算的函数,呈现精确的幂律关系。

1.1 Summary

主要结论概述

Our key findings for Transformer language models are as follows.

本文关于 Transformer 语言模型的关键发现如下。

Performance depends strongly on scale, weakly on model shape. Model performance depends most strongly on scale, which consists of three factors: the number of model parameters N (excluding embeddings), the size of the dataset D , and the amount of compute C used for training. Within reasonable limits, performance depends very weakly on other architectural hyperparameters such as depth vs. width.

性能强烈依赖于尺度,微弱依赖于模型形状。模型性能最强烈地依赖于尺度,它由三个因素组成:模型参数量 N (不含嵌入)、数据集规模 D ,以及训练所用算力 C 。在合理范围内,深度与宽度等其它架构超参数对性能的影响非常微弱。

Smooth power laws. Performance has a power-law relationship with each of the three scale factors N , D , C when not bottlenecked by the other two, with trends spanning more than six orders of magnitude. We observe no signs of deviation from these trends on the upper end, though performance must flatten out eventually before reaching zero loss.

平滑的幂律。在不因其它两者而成为瓶颈的前提下,性能与 N 、 D 、 C 三个尺度因子各自呈幂律关系,趋势跨越超过六个数量级。在上限附近,我们没有观察到这些趋势出现偏离的迹象,尽管性能必然会在损失为零之前趋于平缓。

Universality of overfitting. Performance improves predictably as long as we scale up N and D in tandem, but enters a regime of diminishing returns if either N or D is held fixed while the other increases. The performance penalty depends predictably on the ratio $N^{0.74} / D$, meaning that every time we increase the model size $8\times$, we only need to increase the data by roughly $5\times$ to avoid a penalty.

过拟合的普适性。只要 N 与 D 同步增大,性能就会可预测地提升;但若其中一个固定而另一个增大,则会进入收益递减的区域。性能惩罚可由比值 $N^{0.74} / D$ 可预测地刻画,这意味着:每当模型规模扩大 8 倍,我们只需将数据扩大约 5 倍即可避免过拟合惩罚。

Universality of training. Training curves follow predictable power-laws whose parameters are roughly independent of the model size. By extrapolating the early part of a training curve, we can roughly predict the loss that would be achieved if we trained for much longer.

训练过程的普适性。训练曲线遵循可预测的幂律,其参数与模型规模大致无关。通过对训练曲线早期部分进行外推,我们可以粗略预测在更长时间训练下能达到的损失。

Transfer improves with test performance. When we evaluate models on text with a different distribution than they were trained on, the results are strongly correlated to those on the training validation set with a roughly constant offset in the loss — in other words, transfer to a different distribution incurs a constant penalty but otherwise improves roughly in line with performance on the training set.

迁移随测试性能提升而提升。当我们在与训练数据分布不同的文本上评估模型时,结果与训练验证集上的结果强相关,损失的偏移大致为常数——换言之,迁移到不同分布会带来一个常数惩罚,除此之外,其提升大致与训练集上的性能同步。

Sample efficiency. Large models are more sample-efficient than small models, reaching the same level of performance with fewer optimization steps and using fewer data points.

样本效率。更大的模型比小模型更加样本高效:它们可以用更少的优化步数、更少的数据点达到相同的性能水平。

Convergence is inefficient. When working within a fixed compute budget C but without any other restrictions on the model size N or available data D , we attain optimal performance by training very large models and stopping significantly short of convergence. Maximally compute-efficient training would therefore be far more sample efficient than one might expect based on training small models to convergence, with data requirements growing very slowly as $D \sim C^{0.27}$ with training compute.

收敛是低效的。在固定的算力预算 C 下,若不对模型规模 N 或可用数据 D 施加其它约束,我们应当训练非常大的模型,并在远未收敛时就提前停止训练。因此,算力最优的训练比基于训练小模型至收敛的预期要样本高效得多,数据需求随训练算力增长得非常缓慢,大致为 $D \propto C^{\{0.27\}}$ 。

Optimal batch size. The ideal batch size for training these models is roughly a power of the loss only, and continues to be determinable by measuring the gradient noise scale; it is roughly 1–2 million tokens at convergence for the largest models we can train.

最优批大小。这些模型训练的理想批大小大致仅由损失的某个幂次决定,并且仍然可以通过梯度噪声规模来确定;在我们所能训练的最大模型上,收敛时的批大小约为 100 – 200 万个标记。

Taken together, these results show that language modeling performance improves smoothly and predictably as we appropriately scale up model size, data, and compute. We expect that larger language models will perform better and be more sample efficient than current models.

综上,这些结果表明:只要我们适当增大模型规模、数据和算力,语言建模性能就会平滑且可预测地提升。我们预期,更大的语言模型将比当前模型表现更好、且更样本高效。

1.2 Summary of Scaling Laws

尺度定律汇总

The test loss of a Transformer trained to autoregressively model language can be predicted using a power-law when performance is limited by only either the number of non-embedding parameters N , the dataset size D , or the optimally allocated compute budget $C_{\min}$:

当性能仅受到非嵌入参数量 N 、数据集规模 D 或最优算力预算 $C_{\min}$ 其中之一的限制时,可以用幂律来预测自回归语言 Transformer 的测试损失:

$$L(N) = (N_c / N)^{\{\alpha_N\}}, \alpha_N \approx 0.076, N_c \approx 8.8 \times 10^{13} \text{ (non-embed params)}$$

$$L(D) = (D_c / D)^{\{\alpha_D\}}, \alpha_D \approx 0.095, D_c \approx 5.4 \times 10^{13} \text{ tokens}$$

$$L(C_{\min}) = (C_c^{\min} / C_{\min})^{\{\alpha_{C^{\min}}\}}, \alpha_{C^{\min}} \approx 0.050, C_c^{\min} \approx 3.1 \times 10^8 \text{ PF-days}$$

$$L(N) = (N_c / N)^{\{\alpha_N\}}, \alpha_N \approx 0.076, N_c \approx 8.8 \times 10^{13} \text{ (非嵌入参数量)}$$

$$L(D) = (D_c / D)^{\{\alpha_D\}}, \alpha_D \approx 0.095, D_c \approx 5.4 \times 10^{13} \text{ tokens}$$

$$L(C_{\min}) = (C_c^{\min} / C_{\min})^{\{\alpha_{C^{\min}}\}}, \alpha_{C^{\min}} \approx 0.050, C_c^{\min} \approx 3.1 \times 10^8 \text{ PF-days}$$

These relations hold across eight orders of magnitude in $C_{\min}$, six orders of magnitude in N , and over two orders of magnitude in D . They depend very weakly on model shape and other Transformer hyperparameters (depth, width, number of self-attention heads), with specific numerical values associated with the WebText2 training set. The power laws α_N , α_D , $\alpha_{C^{\min}}$ specify the degree of performance improvement expected as we scale up N , D , or $C_{\min}$; for example, doubling the number of parameters yields a loss that is smaller by a factor $2^{\{-\alpha_N\}} \approx 0.95$. The precise numerical values of N_c , $C_c^{\min}$, and D_c depend on the vocabulary size and tokenization and hence do not have a fundamental meaning.

这些关系在 $C_{\min}$ 上跨越 8 个数量级、在 N 上跨越 6 个数量级、在 D 上跨越 2 个数量级。它们几乎与模型形状及其它 Transformer 超参数(深度、宽度、自注意力头数)无关,具体数值与 WebText2 训练集相对应。幂律指数 α_N 、 α_D 、 $\alpha_{C^{\min}}$ 刻画了随 N 、 D 、 $C_{\min}$ 扩大而带来的性能提升程度;例如,参数量翻倍,损失减小 $2^{\{-\alpha_N\}} \approx 0.95$ 倍。 N_c 、 $C_c^{\min}$ 、 D_c 的具体数值依赖于词表大小和分词方式,故并无根本性意义。

The critical batch size, which determines the speed/efficiency tradeoff for data parallelism, also roughly obeys a power law in L : $B_{\text{crit}}(L) = B_* / L^{\{1/\alpha_B\}}$, with $B_* \approx 2 \times 10^8$ tokens and $\alpha_B \approx 0.21$.

决定数据并行速度/效率权衡的临界批大小,也大致服从关于 L 的幂律: $B_{\text{crit}}(L) = B_* / L^{\{1/\alpha_B\}}$, 其中 $B_* \approx 2 \times 10^8$ tokens, $\alpha_B \approx 0.21$ 。

Equations (1) and (2) together suggest that as we increase the model size, we should increase the dataset size sublinearly according to $D \propto N^{\{\alpha_N / \alpha_D\}} \sim N^{\{0.74\}}$. In fact, we find that there is a single equation combining (1) and (2) that governs the simultaneous dependence on N and D and governs the degree of overfitting:

公式 (1) 与 (2) 共同表明:随着模型规模增大,我们应当按 $D \propto N^{\{\alpha_N / \alpha_D\}} \sim N^{\{0.74\}}$

这种次线性关系增加数据。事实上,我们找到了一个同时刻画 N 与 D 依赖关系并刻画过拟合程度的统一公式:

$$L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$$

$$L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$$

with fits pictured on the left in Figure 2. We conjecture that this functional form may also parameterize the trained log-likelihood for other generative modeling tasks.

其拟合结果如图 2 左侧所示。我们猜想,这一函数形式也可能刻画其它生成式建模任务中训练得到的对数似然。

When training a given model for a finite number of parameter update steps S in the infinite data limit, after an initial transient period, the learning curves can be accurately fit by $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{\min}(S))^{\alpha_S}$, where $S_c \approx 2.1 \times 10^3$ and $\alpha_S \approx 0.76$, and $S_{\min}(S)$ is the minimum possible number of optimization steps estimated using the adjusted-steps formula.

在无穷数据极限下,给定模型经有限步参数更新 S 训练时,经过初始的瞬态期后,学习曲线可以被 $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{\min}(S))^{\alpha_S}$ 精确拟合,其中 $S_c \approx 2.1 \times 10^3$, $\alpha_S \approx 0.76$, $S_{\min}(S)$ 是由调整后的步数公式估计出的最优化步数。

When training within a fixed compute budget C , but with no other constraints, the equation above leads to the prediction that the optimal model size N , optimal batch size B , optimal number of steps S , and dataset size D should grow as $N \propto C^{\alpha_C^{\min} / \alpha_N}$, $B \propto C^{\alpha_C^{\min} / \alpha_B}$, $S \propto C^{\alpha_C^{\min} / \alpha_S}$, $D = B \cdot S$, with $\alpha_C^{\min} = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N)$, which closely matches the empirically optimal results $N \propto C_{\min}^{0.73}$, $B \propto C_{\min}^{0.24}$, and $S \propto C_{\min}^{0.03}$. As the computational budget C increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size.

在固定算力预算 C 下,若不施加其它约束,上述方程可预测最优模型规模 N 、最优批大小 B 、最优步数 S 以及数据集规模 D 应满足: $N \propto C^{\alpha_C^{\min} / \alpha_N}$, $B \propto C^{\alpha_C^{\min} / \alpha_B}$, $S \propto C^{\alpha_C^{\min} / \alpha_S}$, $D = B \cdot S$, 其中 $\alpha_C^{\min} = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N)$ 。这与经验最优结果 $N \propto C_{\min}^{0.73}$ 、 $B \propto C_{\min}^{0.24}$ 、 $S \propto C_{\min}^{0.03}$ 高度吻合。随着算力预算 C 的增加,应主要投入到更大的模型上,而训练时间或数据集规模则不必大幅增加。

1.3 Notation

记号说明

We use the following notation:

本文使用的记号如下:

- L — the cross entropy loss in nats. Typically averaged over the tokens in a context, but in some cases we report the loss for specific tokens within the context.
- N — the number of model parameters, excluding all vocabulary and positional embeddings.
- $C \approx 6 N B S$ — an estimate of the total non-embedding training compute, where B is the batch size, and S is the number of training steps (parameter updates). We quote numerical values in PF-days (1 PF-day = 8.64×10^{19} FLOPs).
- D — the dataset size in tokens.
- B_{crit} — the critical batch size, defined and discussed in Section 5.1. Training at B_{crit} provides a roughly optimal compromise between time and compute efficiency.
- $C_{\min}$ — an estimate of the minimum amount of non-embedding compute to reach a given value of the loss.
- $S_{\min}$ — an estimate of the minimal number of training steps needed to reach a given value of the loss.
- α_X — power-law exponents for the scaling of the loss as $L(X) \propto 1 / X^{\alpha_X}$ where X can be any of $N, D, C, S, B, C_{\min}$.
 - L — 交叉熵损失(以 nats 为单位)。通常取上下文内标记上的平均值;但在某些情况下,会报告上下文中特定位置的损失。
 - N — 模型参数数量,不包括所有词嵌入与位置嵌入。
 - $C \approx 6 N B S$ — 非嵌入训练算力的估计值,其中 B 为批大小, S 为训练步数(即参数更新次数)。我们以 PF-days 为单位给出数值(1 PF-day = 8.64×10^{19} 次浮点运算)。
 - D — 数据集规模(以 token 为单位)。
 - B_{crit} — 临界批大小,定义与讨论见第 5.1 节。在 B_{crit} 下训练大致能在时间与算力效率之间取得最优折中。
 - $C_{\min}$ — 达到给定损失值所需的最小非嵌入算力的估计。
 - $S_{\min}$ — 达到给定损失值所需的最少训练步数的估计。

- α_X —— 损失尺度关系的幂律指数,即 $L(X) \propto 1/X^{\alpha_X}$,其中 X 可以是 N 、 D 、 C 、 S 、 B 、 $C_{\min}$ 中的任意一个。

2. Background and Methods

背景与方法

We train language models on WebText2, an extended version of the WebText dataset, tokenized using byte-pair encoding with a vocabulary size $n_{\text{vocab}} = 50257$. We optimize the autoregressive log-likelihood (i.e. cross-entropy loss) averaged over a 1024-token context, which is also our principal performance metric. We record the loss on the WebText2 test distribution and on a selection of other text distributions. We primarily train decoder-only Transformer models, though we also train LSTM models and Universal Transformers for comparison.

我们在 WebText2(扩展版 WebText 数据集)上训练语言模型,使用字节对编码 (BPE) 进行分词,词表大小 $n_{\text{vocab}} = 50257$ 。我们优化 1024 token 上下文上的自回归对数似然(即交叉熵损失),这也是我们主要的性能指标。我们在 WebText2 测试集分布以及若干其它文本分布上记录损失。我们主要训练仅含解码器的 Transformer 模型,同时为对比也训练了 LSTM 模型与 Universal Transformer。

2.1 Parameter and Compute Scaling of Transformers

Transformer 的参数与算力尺度

We parameterize the Transformer architecture using hyperparameters n_{layer} (number of layers), d_{model} (dimension of the residual stream), d_{ff} (dimension of the intermediate feed-forward layer), d_{attn} (dimension of the attention output), and n_{heads} (number of attention heads per layer). We include n_{ctx} tokens in the input context, with $n_{\text{ctx}} = 1024$ except where otherwise noted.

我们用以下超参数刻画 Transformer

架构: n_{layer} (层数)、 d_{model} (残差流维度)、 d_{ff} (中间前馈层维度)、 d_{attn} (注意力输出维度)以及 n_{heads} (每层注意力头数)。输入上下文包含 n_{ctx} 个 token,除特别说明外, $n_{\text{ctx}} = 1024$ 。

We use N to denote the model size, which we define as the number of non-embedding parameters: $N \approx 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) = 12 n_{\text{layer}} d_{\text{model}}^2$ with the standard $d_{\text{attn}} = d_{\text{ff}} / 4 = d_{\text{model}}$, where we have excluded biases and other sub-leading terms. Our models also have $n_{\text{vocab}} \cdot d_{\text{model}}$ parameters in an embedding matrix, and use $n_{\text{ctx}} \cdot d_{\text{model}}$ parameters for positional embeddings, but we do not include these when discussing the model size N ; we will see that this produces significantly cleaner scaling laws.

我们用 N 表示模型规模,定义其为非嵌入参数量: $N \approx 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) = 12 n_{\text{layer}} d_{\text{model}}^2$ (在标准设置 $d_{\text{attn}} = d_{\text{ff}} / 4 = d_{\text{model}}$ 下),其中已排除偏置及其它次要项。模型还有 $n_{\text{vocab}} \cdot d_{\text{model}}$ 个词嵌入矩阵参数,以及 $n_{\text{ctx}} \cdot d_{\text{model}}$ 个位置嵌入参数;但在讨论模型规模 N 时,我们不计入这些参数——之后我们会看到,这样做能带来显著更干净的尺度定律。

Evaluating a forward pass of the Transformer involves roughly $C_{\text{forward}} \approx 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{model}}$ add-multiply operations, where the factor of two comes from the multiply-accumulate operation used in matrix multiplication. A more detailed per-operation parameter and compute count is included in Table 1. For contexts and models with $d_{\text{model}} > n_{\text{ctx}} / 12$, the context-dependent computational cost per token is a relatively small fraction of the total compute. Since we primarily study models where $d_{\text{model}} \gg n_{\text{ctx}} / 12$, we do not include context-dependent terms in our training compute estimate. Accounting for the backwards pass (approximately twice the compute as the forwards pass), we then define the estimated non-embedding compute as $C \approx 6 N$ floating point operators per training token.

评估 Transformer 的一次前向传播大约需要 $C_{\text{forward}} \approx 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{model}}$ 次乘加运算(其中因子 2 来自矩阵乘法中的乘累加操作)。更详细的每操作参数与算力计数见表 1。对于 $d_{\text{model}} > n_{\text{ctx}} / 12$ 的上下文与模型,与上下文相关的每 token 计算成本在总算力中只占很小一部分。由于我们主要研究 d_{model} 远大于 $n_{\text{ctx}} / 12$ 的模

型,因此在训练算力的估计中不计与上下文相关的项。考虑反向传播(算力约为前向传播的两倍),我们将估计的非嵌入算力定义为每训练 token 约 $C \approx 6 N$ 次浮点运算。

Operation / ■■	Parameters / ■■■■	FLOPs per Token / ■ token FLOPs
Embed	$(n_{\text{vocab}} + n_{\text{ctx}}) \cdot d_{\text{model}}$	$4 d_{\text{model}}$

Attention: QKV	$n_{\text{layer}} \cdot d_{\text{model}} \cdot 3 d_{\text{attn}}$	$2 n_{\text{layer}} \cdot d_{\text{model}} \cdot 3 d_{\text{attn}}$
Attention: Mask	—	$2 n_{\text{layer}} \cdot n_{\text{ctx}} \cdot d_{\text{attn}}$
Attention: Project	$n_{\text{layer}} \cdot d_{\text{attn}} \cdot d_{\text{model}}$	$2 n_{\text{layer}} \cdot d_{\text{attn}} \cdot d_{\text{model}}$
Feedforward	$n_{\text{layer}} \cdot 2 d_{\text{model}} \cdot d_{\text{ff}}$	$2 n_{\text{layer}} \cdot 2 d_{\text{model}} \cdot d_{\text{ff}}$
De-embed	—	$2 d_{\text{model}} \cdot n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{fwd}} = 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

Table 1 — Parameter counts and compute (forward pass) estimates for a Transformer model. Sub-leading terms such as nonlinearities, biases, and layer normalization are omitted.

1 — Transformer

2.2 Training Procedures

训练流程

Unless otherwise noted, we train models with the Adam optimizer for a fixed 2.5×10^5 steps with a batch size of 512 sequences of 1024 tokens. Due to memory constraints, our largest models (more than 1B parameters) were trained with Adafactor. We experimented with a variety of learning rates and schedules, as discussed in the Appendix. We found that results at convergence were largely independent of learning rate schedule. Unless otherwise noted, all training runs included in our data used a learning rate schedule with a 3000 step linear warmup followed by a cosine decay to zero.

除非另有说明,我们使用 Adam 优化器进行固定 2.5×10^5 步的训练,批大小为 512 个长度为 1024 token 的序列。受显存限制,我们最大的模型(超过 10 亿参数)使用 Adafactor 训练。我们尝试了多种学习率与学习率调度(详见附录)。我们发现,收敛结果几乎与学习率调度无关。除非另有说明,数据中包含的所有训练运行均使用 3000 步线性 warmup 之后接余弦衰减至零的学习率调度。

2.3 Datasets

数据集

We train our models on an extended version of the WebText dataset. The original WebText dataset was a web scrape of outbound links from Reddit through December 2017 which received at least 3 karma. In WebText2, we added outbound Reddit links from January to October 2018, also with a minimum of 3 karma. The karma threshold served as a heuristic for whether people found the link interesting or useful. The text of the new links was extracted with the Newspaper3k python library. In total, the dataset consists of 20.3M documents containing 96 GB of text and 1.62×10^{11} words. We then apply the reversible tokenizer, which yields 2.29×10^{11} tokens. We reserve 6.6×10^8 of these tokens for use as a test set, and we also test on similarly-prepared samples of Books Corpus, Common Crawl, English Wikipedia, and a collection of publicly-available Internet Books.

我们在 WebText 数据集的扩展版上训练模型。原始 WebText 是从 Reddit 中抽取的外链网页抓取,截至 2017 年 12 月,要求每条链接至少获得 3 个 karma。在 WebText2 中,我们新增了 2018 年 1 月至 10 月的 Reddit 外链,同样要求至少 3 个 karma。karma 阈值作为一条启发式规则,用以判断链接是否对人们有趣或有用。新增链接的文本通过 Newspaper3k 这个 Python 库抽取。整体而言,数据集共包含 2030 万篇文档、96 GB 文本、 1.62×10^{11} 个词。然后我们应用可逆的分词器,得到 2.29×10^{11} 个 token。我们将其中 6.6×10^8 个 token 留作测试集,同时还在 Books Corpus、Common Crawl、英文维基百科以及一组公开可用的 Internet Books 的同种处理样本上进行测试。

3. Empirical Results and Basic Power Laws

实验结果与基本幂律

To characterize language model scaling we train a wide variety of models, varying a number of factors including: model size (ranging in size from 768 to 1.5 billion non-embedding parameters); dataset size (ranging from 22 million to 23 billion tokens); shape (including depth, width, attention heads, and feed-forward dimension); context length (1024 for most runs, though we also experiment with shorter contexts); and batch size (2^{11} for most runs, but we also vary it to measure the critical batch size).

为刻画语言模型的尺度,我们在多种因素上变化训练:模型规模(从 768 到 15 亿非嵌入参数);数据集规模(从 200 万到 230 亿 token);形状(深度、宽度、注意力头、前馈维度);上下文长度(大多数运行使用 1024,同时也实验了更短的上下文);批大小(大多数运行使用 2^{19} ,但我们也改变它以测量临界批大小)。

In this section we will display data along with empirically-motivated fits, deferring theoretical analysis to later sections. 本节我们将展示数据及由经验驱动的拟合,而将理论分析留到后续章节。

3.1 Approximate Transformer Shape and Hyperparameter Independence

Transformer 形状与超参数近似无关性

Transformer performance depends very weakly on the shape parameters n_{layer} , n_{heads} , and d_{ff} when we hold the total non-embedding parameter count N fixed. To establish these results we trained models with fixed size while varying a single hyperparameter. This was simplest for the case of n_{heads} . When varying n_{layer} , we simultaneously varied d_{model} while keeping $N \approx 12 n_{\text{layer}} d_{\text{model}}^2$ fixed. Similarly, to vary d_{ff} at fixed model size we also simultaneously varied the d_{model} parameter, as required by the parameter counts in Table 1. Independence of n_{layers} would follow if deeper Transformers effectively behave as ensembles of shallower models, as has been suggested for ResNets.

在保持总非嵌入参数量 N 不变的情况下,Transformer 的性能几乎不依赖于形状超参数 n_{layer} 、 n_{heads} 和 d_{ff} 。为建立这一结论,我们在固定规模的同时,逐个改变某一个超参数进行了实验。最直接的是 n_{heads} 。改变 n_{layer} 时,我们同时调整 d_{model} 以保持 $N \approx 12 n_{\text{layer}} d_{\text{model}}^2$ 不变。类似地,在固定 N 下改变 d_{ff} 时,我们也同时改变 d_{model} (如表 1 中的参数量公式所要求的)。如果更深的 Transformer 实际上表现为若干浅层模型的集成(有人曾对 ResNet 提出过类似观点),那么 n_{layer} 的无关性也就顺理成章了。

3.2 Performance with Non-Embedding Parameter Count N

性能与非嵌入参数量 N 的关系

We display the performance of a wide variety of models, ranging from small models with shape $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$ through billion-parameter models, ranging in shape from $(6, 4288)$ through $(207, 768)$. Here we have trained to near convergence on the full WebText2 dataset and observe no overfitting (except possibly for the very largest models).

我们展示了大量模型的性能——从形状 $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$ 的小模型,到形状从 $(6, 4288)$ 到 $(207, 768)$ 的十亿参数模型。我们是在完整的 WebText2 数据集上训练至接近收敛的,且没有观察到过拟合(最大的几个模型可能除外)。

We find a steady trend with non-embedding parameter count N , which can be fit to the first term of the $L(N, D)$ equation, so that $L(N) \approx (N_c / N)^{\alpha_N}$. To observe these trends it is crucial to study performance as a function of N ; if we instead use the total parameter count (including the embedding parameters) the trend is somewhat obscured. This suggests that the embedding matrix can be made smaller without impacting performance, as has been seen in recent work.

我们发现非嵌入参数量 N 与性能呈现稳定的趋势,可以由 $L(N, D)$ 公式的第一项拟合,即 $L(N) \approx (N_c / N)^{\alpha_N}$ 。要观察这一趋势,必须以 N 为自变量来研究性能;若使用总参数量(含嵌入参数),趋势会变得模糊。这说明词嵌入矩阵可以做得更小而不影响性能——近期工作中也观察到了这一点。

Although these models have been trained on the WebText2 dataset, their test loss on a variety of other datasets is also a power-law in N with nearly identical power, as shown in Figure on generalization vs model size.

尽管这些模型只在 WebText2 上训练,但它们在多种其它数据集上的测试损失同样与 N 呈幂律关系,且幂律指数几乎相同(参见泛化与模型规模图)。

3.3 Performance with Dataset Size and Compute

性能与数据集规模、算力的关系

We display empirical trends for the test loss as a function of dataset size D (in tokens) and training compute C in Figure on basic power laws. For the trend with D we trained a model with $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$ on fixed subsets of the WebText2 dataset. We stopped training once the test loss ceased to decrease. We see that the resulting test losses can be fit with simple power-law $L(D) \approx (D_c / D)^{\alpha_D}$ in the dataset size.

我们在基本幂律图中展示了测试损失作为数据集规模 D (以 token 计)和训练算力 C 的函数所呈现的经验趋势。在与 D 的关系上,我们在 WebText2 的固定子集上训练了一个形状为 $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$ 的模型,并在测试损失不再下降时停止训练。我们发现,所得的测试损失可由 $L(D) \approx$

$(D_c / D)^{\alpha_D}$ 这一简单的幂律拟合。

The total amount of non-embedding compute used during training can be estimated as $C = 6 N B S$, where B is the batch size, S is the number of parameter updates, and the factor of 6 accounts for the forward and backward passes. Thus for a given value of C we can scan over all models with various N to find the model with the best performance on step $S = C / (6 B S)$. Note that in these results the batch size B remains fixed for all models, which means that these empirical results are not truly optimal. We will account for this in later sections using an adjusted $C_{\min}$ to produce cleaner trends. The result can be fit with $L(C) \approx (C_c / C)^{\alpha_C}$. The data strongly suggests that sample efficiency improves with model size.

训练过程中使用的总非嵌入算力可估计为 $C = 6 N B S$, 其中 B 为批大小, S 为参数更新次数, 因子 6 涵盖了前向与反向传播。因此, 对于给定的 C , 我们可以在不同 N 的模型上扫描, 找到在 $S = C / (6 B S)$ 步上性能最佳的模型。需要注意的是, 在这些结果中, 所有模型使用的批大小 B 都是固定的, 这意味着这些经验结果并非真正最优。我们将在后续章节中通过调整后的 $C_{\min}$ 来解释这一点, 从而得到更干净的趋势。结果可由 $L(C) \approx (C_c / C)^{\alpha_C}$ 拟合。数据强烈表明: 样本效率随模型规模的增大而提升。

4. Charting the Infinite Data Limit and Overfitting

刻画无穷数据极限与过拟合

In Section 3 we found a number of basic scaling laws for language modeling performance. Here we will study the performance of a model of size N trained on a dataset with D tokens while varying N and D simultaneously. We will empirically demonstrate that the optimally trained test loss accords with the scaling law $L(N, D)$. This provides guidance on how much data we would need to train models of increasing size while keeping overfitting under control.

在第 3 节中, 我们发现了若干基本的语言建模性能尺度定律。本节中, 我们将研究在同时变化 N 与 D 时, 一个规模为 N 、在 D 个 token 数据集上训练的模型的性能。我们将以经验方式证明: 经过最优训练的测试损失服从 $L(N, D)$ 这一尺度定律。这为我们在控制过拟合的前提下训练不断增大的模型所需的数据量提供了指引。

4.1 Proposed $L(N, D)$ Equation

提出的 $L(N, D)$ 公式

We have chosen the parameterization $L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$ using three principles:

我们采用 $L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$ 这一参数形式, 基于三条原则:

- Changes in vocabulary size or tokenization are expected to rescale the loss by an overall factor. The parameterization of $L(N, D)$ (and all models of the loss) must naturally allow for such a rescaling.
- Fixing D and sending $N \rightarrow \infty$, the overall loss should approach $L(D)$. Conversely, fixing N and sending $D \rightarrow \infty$ the loss must approach $L(N)$.
- $L(N, D)$ should be analytic at $D = \infty$, so that it has a series expansion in $1/D$ with integer powers. Theoretical support for this principle is significantly weaker than for the first two.
 - 词表大小或分词方式的变化预期只会对损失乘以一个整体常数。因此, $L(N, D)$ (以及所有刻画损失的模型) 的参数形式必须自然地允许这种整体缩放。
 - 固定 D 并令 $N \rightarrow \infty$, 整体损失应趋于 $L(D)$ 。反过来, 固定 N 并令 $D \rightarrow \infty$, 损失必须趋于 $L(N)$ 。
 - $L(N, D)$ 应当在 $D = \infty$ 处解析, 从而在 $1/D$ 上可做整数次幂的级数展开。这一原则的理论支持显著弱于前两条。

Our choice of $L(N, D)$ satisfies the first requirement because we can rescale N_c, D_c with changes in the vocabulary. This also implies that the values of N_c, D_c have no fundamental meaning. Since we stop training early when the test loss ceases to improve and optimize all models in the same way, we expect that larger models should always perform better than smaller models. But with fixed finite D , we also do not expect any model to be capable of approaching the best possible loss (i.e. the entropy of text). Similarly, a model with fixed size will be capacity-limited. These considerations motivate our second principle. Knowledge of $L(N)$ at infinite D and $L(D)$ at infinite N fully determines all the parameters in $L(N, D)$. The third principle explains the asymmetry between the roles of N and D in the equation. Very similar symmetric expressions are possible, but they would not have a $1/D$ expansion with integer powers, and would require the introduction of an additional parameter. In any case, we will see that our equation for $L(N, D)$ fits the data well, which is the most important justification for our ansatz.

我们选择的 $L(N, D)$ 满足第一条要求,因为我们可以通过词表变化对 N_c 、 D_c 进行缩放。这也意味着 N_c 、 D_c 的具体数值并无根本意义。由于我们在测试损失不再下降时提前停止训练,并以相同方式优化所有模型,我们预期更大的模型总是优于更小的模型;但在固定有限 D 下,我们不预期任何模型能达到可能的最佳损失(即文本的熵)。类似地,固定规模的模型存在容量上限。这些考量是第二条原则的依据。在无穷 D 处的 $L(N)$ 与在无穷 N 处的 $L(D)$ 完全确定了 $L(N, D)$ 中的所有参数。第三条原则解释了公式中 N 与 D 之间的不对称性。类似的、形式上更对称的表达式是存在的,但它们在 $1/D$ 上没有整数次幂的展开,且需要引入额外的参数。无论如何,我们将看到, $L(N, D)$ 公式能很好地拟合数据,这是我们采用这一假设的最重要依据。

4.2 Results

实验结果

We regularize all our models with 10% dropout, and by tracking test loss and stopping once it is no longer decreasing. The results are displayed in Figure 3, including a fit to the four parameters α_N , α_D , N_c , D_c in $L(N, D)$: $\alpha_N = 0.076$, $\alpha_D = 0.103$, $N_c = 6.4 \times 10^{13}$, $D_c = 1.8 \times 10^{13}$.

我们对所有模型使用 10% 的 dropout,并通过追踪测试损失、待其不再下降时停止训练。拟合 $L(N, D)$ 中四个参数 α_N 、 α_D 、 N_c 、 D_c 的结果如图 3 所示: $\alpha_N = 0.076$, $\alpha_D = 0.103$, $N_c = 6.4 \times 10^{13}$, $D_c = 1.8 \times 10^{13}$ 。

We obtain an excellent fit, with the exception of the runs where the dataset has been reduced by a factor of 1024, to about 2×10^7 tokens. With such a small dataset, an epoch consists of only 40 parameter updates. Perhaps such a tiny dataset represents a different regime for language modeling, as overfitting happens very early in training.

除了将数据集缩减为原来的 $1/1024$ (约 2×10^7 token)的那些运行外,我们都得到了优秀的拟合。这么小的数据集,一个 epoch 仅包含 40 次参数更新。或许这种极小的数据集代表语言建模的一种不同模式,因为过拟合在训练初期就会发生。

To chart the borderlands of the infinite data limit, we can directly study the extent of overfitting. For all but the largest models, we see no sign of overfitting when training with the full 22B token WebText2 dataset, so we can take it as representative of $D = \infty$. Thus we can compare finite D to the infinite data limit by defining $\delta L(N, D) \equiv L(N, D) / L(N, \infty) - 1$ and studying it as a function of N , D . We see empirically that δL depends only a specific combination of N and D , as shown in the appendix.

为了刻画无穷数据极限的边界,我们可以直接研究过拟合的程度。除最大的模型之外,在完整 220 亿 token 的 WebText2 数据集上训练时,我们看不到任何过拟合迹象,因此可以把它视为 $D = \infty$ 的代表。我们可以定义 $\delta L(N, D) \equiv L(N, D) / L(N, \infty) - 1$,以此来比较有限 D 与无穷数据极限。经验上我们发现, δL 只依赖于 N 与 D 的某个特定组合(详见附录)。

We estimate that the variation in the loss with different random seeds is roughly 0.02, which means that to avoid overfitting when training to within that threshold of convergence we require $D \gtrsim (5 \times 10^3) \cdot N^{0.74}$. With this relation, models smaller than 10^9 parameters can be trained with minimal overfitting on the 22B token WebText2 dataset, but our largest models will encounter some mild overfitting. More generally, this relation shows that dataset size may grow sub-linearly in model size while avoiding overfitting. Note however that this does not typically represent maximally compute-efficient training. We should also emphasize that we have not optimized regularization (e.g. the dropout probability) while varying dataset and model size.

我们估计,在不同随机种子下损失的变化约为 0.02,因此为了在训练至与收敛相差此阈值时避免过拟合,我们需要 $D \gtrsim (5 \times 10^3) \cdot N^{0.74}$ 。按此关系,在 220 亿 token 的 WebText2 上训练 10^9 参数以下的模型几乎不会出现过拟合,但我们的最大模型会遭遇一定程度的轻微过拟合。更一般地,这一关系表明:为避免过拟合,数据集规模可以按模型规模的次线性增长。不过,这种关系通常并不代表算力最优的训练方式。我们还须强调,在变化数据集与模型规模时,我们并未优化正则化(如 dropout 概率)。

5. Scaling Laws with Model Size and Training Time

模型规模与训练时间的尺度定律

In this section we will demonstrate that a simple scaling law provides a good description for the loss as a function of model size N and training time. First we will explain how to use previous results to define a universal training step $S_{\min}$, which accounts for the fact that most of our models have not been trained at an optimal batch size. Then we will demonstrate that we can fit the model size and training time dependence of the loss using $L(N, S)$. Later we will use these results to predict the optimal allocation of training compute between model size and training time, and then confirm that prediction.

本节中,我们将证明:一个简单的尺度定律能够很好地刻画损失作为模型规模 N 与训练时间的函数。首先,我们将说明如何利用已有结果定义一个普适的训练步数 $S_{\min}$,以解释大多数模型并未在最优批大小下训练这一事实。然后,我们将证明可以用 $L(N, S)$ 来拟合损失对模型规模与训练时间的依赖。后续我们将利用这些结果预测模型规模与训练时间之间的最优算力分配,并通过实验加以验证。

5.1 Adjustment for Training at $B_{\text{crit}}(L)$

按 $B_{\text{crit}}(L)$ 调整训练

A simple empirical theory for the batch size dependence of training was developed in prior work. It was argued that there is a critical batch size B_{crit} for training; for B up to B_{crit} the batch size can be increased with very minimal degradation in compute-efficiency, whereas for $B > B_{\text{crit}}$ increases in B result in diminishing returns. It was also argued that the gradient noise scale provides a simple prediction for B_{crit} , and that neither depends directly on model size except through the value of the loss that has been attained. These results can be used to predict how training time and compute will vary with the batch size. To utilize both training time and compute as effectively as possible, it is best to train with a batch size $B \approx B_{\text{crit}}$.

前人工作发展了一套关于训练中批大小依赖的简单经验理论。该理论认为训练存在一个 临界批大小 B_{crit} :在 $B \leq B_{\text{crit}}$ 的范围内,增大批大小对算力效率的影响极小;而当 $B > B_{\text{crit}}$ 时,继续增大 B 会出现收益递减。该理论还指出,梯度噪声规模为 B_{crit} 提供了一种简单的预测,且两者都只通过损失值间接依赖于模型规模,而不直接依赖于 N 。这些结果可以预测训练时间与算力随批大小的变化。要同时最有效地利用时间与算力,最好在 $B \approx B_{\text{crit}}$ 下进行训练。

More specifically, it was demonstrated that for a wide variety of neural network tasks, the number of training steps S and the number of data examples processed $E = BS$ satisfy the simple relation $(S / S_{\min} - 1)(E / E_{\min} - 1) = 1$ when training to any fixed value of the loss L . Here $S_{\min}$ is the minimum number of steps necessary to reach L , while $E_{\min}$ is the minimum number of data examples that must be processed. This relation defines the critical batch size $B_{\text{crit}}(L) \equiv E_{\min} / S_{\min}$, which is a function of the target value of the loss. Training at the critical batch size makes a roughly optimal time/compute tradeoff, requiring $2 S_{\min}$ training steps and processing $E = 2 E_{\min}$ data examples.

更具体地说,前人证明:对于多种神经网络任务,在训练至任意固定的损失值 L 时,训练步数 S 与处理的数据样本数 $E = BS$ 满足简单关系 $(S / S_{\min} - 1)(E / E_{\min} - 1) = 1$ 。其中 $S_{\min}$ 是达到 L 所需的最少步数, $E_{\min}$ 是必须处理的最少样本数。该关系定义了临界批大小 $B_{\text{crit}}(L) \equiv E_{\min} / S_{\min}$,它只是目标损失值的函数。在临界批大小下训练大致能给出一个最优的时间/算力折中:需要 $2 S_{\min}$ 步训练,处理 $E = 2 E_{\min}$ 个样本。

In our experiments we plotted the critical batch size and gradient noise scale as a function of training loss for two different models. We see that $B_{\text{crit}}(L)$ is independent of model size, and only depends on the loss L . So the predictions continue to hold for Transformer language models. The critical batch size can be fit with a power-law in the loss $B_{\text{crit}}(L) \approx B_* / L^{1/\alpha_B}$, where $B_* \approx 2 \times 10^4$ and $\alpha_B \approx 0.21$. We have chosen this parameterization for $B_{\text{crit}}(L)$ because as the loss approaches its minimum value $L_{\min}$, the gradient noise scale is expected to diverge, and we expect B_{crit} to track this noise scale.

在我们的实验中,我们针对两个不同的模型,绘制了临界批大小与梯度噪声规模随训练损失的变化曲线。我们发现, $B_{\text{crit}}(L)$ 与模型规模无关,只依赖于损失 L 。因此,这些预测同样适用于 Transformer 语言模型。临界批大小可由关于损失的幂律拟合: $B_{\text{crit}}(L) \approx B_* / L^{1/\alpha_B}$, 其中 $B_* \approx 2 \times 10^4$, $\alpha_B \approx 0.21$ 。我们之所以选择这种参数形式,是因为当损失接近其最小值 $L_{\min}$ 时,梯度噪声规模预期会发散,我们预期 B_{crit} 会跟踪这一噪声规模。

We will use $B_{\text{crit}}(L)$ to estimate the relation between the number of training steps S while training at batch size $B = 2^i$ tokens and the number of training steps while training at $B \gg B_{\text{crit}}$. This is simply $S_{\min}(S) \equiv S / (1 + B_{\text{crit}}(L) / B)$ (minimum steps, at $B \gg B_{\text{crit}}$) for any given target value L for the loss. This also defines a critical value of the compute needed to train to L with a model of size N if we were to train at $B \ll B_{\text{crit}}(L)$: $C_{\min}(C) \equiv C / (1 + B / B_{\text{crit}}(L))$

(minimum compute, at $B \ll B_{\text{crit}}$), where $C = 6 N B S$ estimates the (non-embedding) compute used at batch size B .

我们将利用 $B_{\text{crit}}(L)$ 来估计在批大小 $B = 2^{19}$ tokens 下训练所需的步数 S 与在 $B \gg B_{\text{crit}}$

下训练所需步数之间的关系。对任意目标损失 L , 这简化为: $S_{\text{min}}(S) \equiv S / (1 + B_{\text{crit}}(L) / B)$ (在 $B \gg B_{\text{crit}}$

下的最少步数)。这也定义了 $B \ll B_{\text{crit}}(L)$ 下训练规模为 N 的模型至损失 L 所需的算力临界值: $C_{\text{min}}(C)$

$\equiv C / (1 + B / B_{\text{crit}}(L))$ (在 $B \ll B_{\text{crit}}$ 下的最少算力), 其中 $C = 6 N B S$ 是在批大小 B

下使用的(非嵌入)算力的估计。

5.2 Results for $L(N, S_{\text{min}})$

$L(N, S_{\text{min}})$ 的结果

Now we will use S_{min} defined above to obtain a simple and universal fit for the dependence of the loss on model size and training time in the infinite data limit. We will fit the stable, Adam-optimized training runs using $L(N, S_{\text{min}}) = (N_{\text{c}} / N)^{\alpha_N} + (S_{\text{c}} / S_{\text{min}})^{\alpha_S}$ for the loss. We include all training steps after the warmup period of the learning rate schedule, and find a fit to the data with the parameters: $\alpha_N = 0.077$, $\alpha_S = 0.76$, $N_{\text{c}} = 6.5 \times 10^{13}$, $S_{\text{c}} = 2.1 \times 10^3$.

现在我们将使用上文定义的 S_{min} , 在无穷数据极限下得到一个简单而普适的、刻画损

失对模型规模与训练时间依赖的拟合。我们用 $L(N, S_{\text{min}}) = (N_{\text{c}} / N)^{\alpha_N} + (S_{\text{c}} / S_{\text{min}})^{\alpha_S}$

对稳定的、由 Adam 优化的训练运行进行拟合。我们只纳入学习率调度 warmup

之后的所有训练步, 并得到参数拟合结果: $\alpha_N = 0.077$, $\alpha_S \approx 0.76$, $N_{\text{c}} = 6.5 \times 10^{13}$, $S_{\text{c}} = 2.1 \times 10^3$ 。

With these parameters, we obtain the learning curve fits. Though the fits are imperfect, we believe they are quite compelling given the simplicity of the equation. The power-law dependence of the loss on S_{min} reflects the interplay of optimizer dynamics and the loss landscape. Since the fits are best late in training, when the loss may be approximately quadratic, the power-law should provide information about the spectrum of the Hessian of the loss. Its universality suggests that the Hessian eigenvalue density is roughly independent of model size.

在这些参数下, 我们得到了学习曲线的拟合结果。尽管拟合并非完美, 但考虑到该方程

的简洁性, 我们认为拟合结果相当有说服力。损失对 S_{min} 的幂律依赖反映了优化器

动力学与损失景观之间的相互作用。由于拟合在训练后期(此时损失近似二次时)效果

最好, 这一幂律应当能提供关于损失 Hessian 谱的信息。其普适性表明 Hessian 特征

值密度大致与模型规模无关。

6. Optimal Allocation of the Compute Budget

算力预算的最优分配

We displayed the empirical trend of performance as a function of the computation used during training in Figure 1. However, this result involved training at a fixed batch size B , whereas we know that in fact we could train more efficiently by training at the batch size B_{crit} discussed above. Large and small values of the loss could have been achieved with fewer samples or fewer steps, respectively, and correcting for this inefficiency by standardizing to the critical batch size results in cleaner and more predictable trends.

我们曾在图 1 中展示性能作为训练所用算力的函数的经验趋势。然而, 这一结果是在固定批大小 B

下训练得到的, 而事实上我们可以通过采用上文讨论的 B_{crit} 来更高效地训

练。更大的损失值本来可以用更少的样本来达到, 更小的损失值本来可以用更少的步数来

达到; 通过标准化到临界批大小来修正这种低效, 可以得到更干净、更可预测的趋势。

In this section we will adjust for this oversight. More importantly, we will use the results of Section 5 to determine the optimal allocation of compute between model size N and the quantity of data processed during training, namely $2 B_{\text{crit}} S_{\text{min}}$. We will determine this allocation both empirically and theoretically, by using the equation for $L(N, S_{\text{min}})$, and we will demonstrate that these methods agree.

本节中我们将修正这一疏漏。更重要的是, 我们将利用第 5 节的结果, 确定模型规模 N

与训练期间所处理的数据量(即 $2 B_{\text{crit}} S_{\text{min}}$)之间的算力最优分配。我们既会从经验

上, 也会从理论上(借助 $L(N, S_{\text{min}})$ 方程)确定这一分配, 并证明两种方法是吻合的。

6.1 Optimal Performance and Allocations

最优性能与分配

Let us first study the loss as a function of the optimally allocated compute from the adjusted compute formula. We see that as compared to the compute plot of Figure 1, the new fit with C_{min} is somewhat improved.

首先,我们研究损失作为由调整后算力公式得到的最优分配算力的函数。我们发现,与图 1 中的算力曲线相比,使用 $C_{\min}$ 的新拟合略有改善。

Given $L(C_{\min})$, it is natural to ask for the optimal model size $N(C_{\min})$ that provides the minimal loss with a given quantity of training compute. The optimal model size can be fit very well with a power-law $N(C_{\min}) \propto C_{\min}^{0.73}$. We show the effect of training models of sub-optimal sizes in the appendix.

有了 $L(C_{\min})$,很自然地会问:在给定训练算力下,能达到最小损失的最优模型规模 $N(C_{\min})$ 是多少?最优模型规模可由幂律 $N(C_{\min}) \propto C_{\min}^{0.73}$ 非常好地拟合。
训练次优尺寸模型的效果见附录。

By definition $C_{\min} \equiv 6 N B_{\text{crit}} S$, and so we can use $N(C_{\min})$ to extract further results. In particular, since prior fits show $B \propto L^{-4.8}$ and $L \propto C_{\min}^{-0.05}$, we can conclude that $B_{\text{crit}} \propto C_{\min}^{0.24}$. This leads us to conclude that the optimal number of steps will only grow very slowly with compute, as $S_{\min} \propto C_{\min}^{0.03}$, matching the empirical results. In fact the measured exponent is sufficiently small that our results may even be consistent with an exponent of zero.

根据定义 $C_{\min} \equiv 6 N B_{\text{crit}} S$,因此我们可以利用 $N(C_{\min})$ 推导出进一步的结果。
特别地,鉴于前文的拟合给出 $B \propto L^{-4.8}$ 和 $L \propto C_{\min}^{-0.05}$,我们可以得到 $B_{\text{crit}} \propto C_{\min}^{0.24}$ 。由此得出最优步数只会随算力增长得非常缓慢: $S_{\min} \propto C_{\min}^{0.03}$,这与经验结果吻合。事实上,测得的指数已经小到可能与零指数一致。

Thus we conclude that as we scale up language modeling with an optimal allocation of computation, we should predominantly increase the model size N , while simultaneously scaling up the batch size via $B \propto B_{\text{crit}}$ with negligible increase in the number of serial steps. Since compute-efficient training uses relatively few optimization steps, additional work on speeding up early training dynamics may be warranted.

因此我们得出结论:在以算力最优分配方式扩展语言建模时,我们应主要增大模型规模 N ,同时通过 $B \propto B_{\text{crit}}$ 增大批大小,而串行步数几乎不必增加。由于算力高效的训练使用的优化步数相对较少,进一步研究如何加速训练早期动力学可能是值得的。

6.2 Predictions from $L(N, S_{\min})$

由 $L(N, S_{\min})$ 给出的预测

The results for $L(C_{\min})$ and the allocations can be predicted from the $L(N, S_{\min})$ equation obtained in Section 5. Given our equation for $L(N, S_{\min})$, we can substitute $S_{\min} = C_{\min} / (6 N B)$ and then find the minimum of the loss as a function of N , while fixing the training compute. For the loss as a function of training compute, we predict that $L(C_{\min}) = (C_{\text{c}}^{\min} / C_{\min})^{\alpha_{\text{C}}^{\min}}$, where $\alpha_{\text{C}}^{\min} \equiv 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.054$, in excellent agreement with the exponent of Figure 4. We also predict that $N(C_{\min}) \propto C_{\min}^{\alpha_{\text{C}}^{\min} / \alpha_N} \approx C_{\min}^{0.71}$, which also matches the scaling of Figure on optimal model size to within a few percent. Our scaling laws provide a predictive framework for the performance of language modeling.

$L(C_{\min})$ 及其分配的结果可由第 5 节得到的 $L(N, S_{\min})$ 方程预测。给定 $L(N, S_{\min})$ 表达式,我们可以代入 $S_{\min} = C_{\min} / (6 N B)$,然后在固定训练算力下求损失关于 N 的极小值。对损失作为训练算力的函数,我们预测为 $L(C_{\min}) = (C_{\text{c}}^{\min} / C_{\min})^{\alpha_{\text{C}}^{\min}}$,其中 $\alpha_{\text{C}}^{\min} \equiv 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.054$,与图 4 中得到的指数吻合得极好。我们还预测 $N(C_{\min}) \propto C_{\min}^{\alpha_{\text{C}}^{\min} / \alpha_N} \approx C_{\min}^{0.71}$,这与最优模型规模图中显示的尺度在百分之几内一致。本文给出的尺度定律为语言建模性能提供了一个可预测的框架。

6.3 Contradictions and a Conjecture

矛盾与猜想

We observe no signs of deviation from straight power-law trends at large values of compute, data, or model size. Our trends must eventually level off, though, since natural language has non-zero entropy. Indeed, the trends for compute-efficient training described in this section already contain an apparent contradiction. At scales several orders of magnitude above those documented here, the performance predicted by the $L(C_{\min})$ scaling law decreases below what should be possible given the slow growth in training data with compute.

在大算力、大数据或大模型处,我们没有观察到直线幂律趋势出现偏离的迹象。然而,这些趋势最终必然趋于平缓,因为自然语言具有非零的熵。事实上,本节所述的算力高效训练趋势已经包含了一个明显的矛盾:在比本文研究范围高几个数量级的尺度上, $L(C_{\min})$ 尺度律所预测的性能会降低到与训练数据随算力缓慢增长所能支持的最低水平之下。

Since the amount of data used by compute-efficient training grows slowly with the compute budget, the performance predicted by $L(C_{\min})$ eventually hits a lower bound set by the $L(D)$ power law. To keep overfitting under control, the results of Section 4 imply that we should scale the dataset size as $D \propto N^{\{0.74\}} \propto C_{\min}^{\{0.54\}}$ where we have used the compute-efficient $N(C_{\min})$.

由于算力高效训练所使用的数据量随算力预算增长缓慢, $L(C_{\min})$ 所预测的性能最终会触及由 $L(D)$ 幂律设定的下界。为控制过拟合, 第 4 节的结果表明数据规模应按 $D \propto N^{\{0.74\}} \propto C_{\min}^{\{0.54\}}$ 增长(其中使用了算力高效的 $N(C_{\min})$)。

Let us compare this to the data requirements of compute-efficient training. If we train at the critical batch size (i.e. $C = 2 C_{\min}$) and never re-use data during training, we find that data usage grows with compute as $D(C_{\min}) \approx (4 \times 10^1 \blacksquare \text{ tokens}) \cdot (C_{\min} / \text{PF-Day})^{\{0.26\}}$. This is the maximum rate at which the dataset size can productively grow with compute, since it means that we are only training for a single epoch. But it grows the dataset much more slowly than in the overfitting relation. It appears to imply that compute-efficient training will eventually run into a problem with overfitting, even if the training process never re-uses any data!

让我们将其与算力高效训练的数据需求作比较。如果我们在临界批大小下训练(即 $C = 2 C_{\min}$), 且训练过程中不重复使用数据, 则可发现数据用量随算力的增长为 $D(C_{\min}) \approx (4 \times 10^1 \blacksquare \text{ tokens}) \cdot (C_{\min} / \text{PF-Day})^{\{0.26\}}$ 。这是数据集规模能够有效地随算力增长的最大速率, 因为这意味着我们仅对数据训练一个 epoch。但其增速仍远慢于前面给出的过拟合关系。这似乎意味着, 即便训练过程从不重复使用数据, 算力高效的训练最终也会遇到过拟合的问题。

According to our basic power laws, we expect that when we are bottlenecked by the dataset size (i.e. by overfitting), the loss should scale as $L(D) \propto D^{\{-0.095\}}$. This implies that the loss would scale with compute as $L(D(C_{\min})) \propto C_{\min}^{\{-0.03\}}$ once we are data-limited. Once again, we have a contradiction, as this will eventually intersect with our prediction for $L(C_{\min})$, where we found a scaling $L(C_{\min}) \propto C_{\min}^{\{-0.050\}}$. The intersection point of $L(D(C_{\min}))$ and $L(C_{\min})$ occurs at $C^* \sim 10 \blacksquare \text{ PF-Days}$, $N^* \sim 10^{12}$ parameters, $D^* \sim 10^{12}$ tokens, $L^* \sim 1.7$ nats/token, though the numerical values are highly uncertain, varying by an order or magnitude in either direction depending on the precise values of the exponents.

根据基本幂律, 我们预期当训练被数据规模(即过拟合)所限制时, 损失应按 $L(D) \propto D^{\{-0.095\}}$ 缩放。这意味着在数据受限情形下, 损失随算力的缩放为 $L(D(C_{\min})) \propto C_{\min}^{\{-0.03\}}$ 。我们再次得到一个矛盾: 它最终会与 $L(C_{\min})$ 的预测($L(C_{\min}) \propto C_{\min}^{\{-0.050\}}$) 相交。 $L(D(C_{\min}))$ 与 $L(C_{\min})$ 的交点出现在 $C^* \approx 10^4 \text{ PF-Days}$ 、 $N^* \approx 10^{12}$ 参数、 $D^* \approx 10^{12}$ tokens、 $L^* \approx 1.7$ nats/token 处; 但具体数值高度不确定, 其数量级可能因指数取值不同而上下波动一个量级。

The most obvious interpretation is that our scaling laws break down at or before we reach this point, which is still many orders of magnitude away in both compute and model size. One might also conjecture that this intersection point has a deeper meaning. If we cannot increase the model size beyond N^* without qualitatively different data requirements, perhaps this means that once we reach $C_{\min}^*$ and N^* , we have extracted all of the reliable information available in natural language data. In this interpretation, L^* would provide a rough estimate for the entropy-per-token of natural language. In this scenario, we would expect the loss trend to level off at or before L^* .

最直接的解释是: 本文的尺度定律会在到达这一交点之前(或恰好在交点处)失效——而该交点在算力和模型规模上都还距离我们许多个数量级。另一种猜想是: 这个交点具有更深层的含义。若我们不能在不对数据要求产生质的改变的前提下把模型规模扩大到 N^* 之外, 那么这或许意味着, 一旦我们达到 $C_{\min}^*$ 与 N^* , 我们就已提取出了自然语言数据中所有可被可靠获取的信息。按照这种解读, L^* 将提供自然语言每 token 熵的一个粗略估计。在此情境下, 我们会预期损失趋势在 L^* 处或之前趋于平缓。

7. Related Work

相关工作

Power laws can arise from a wide variety of sources. Power-law scalings with model and dataset size in density estimation and in random forest models may be connected with our results. These models suggest that power-law exponents may have a very rough interpretation as the inverse of the number of relevant features in the data. Some early work found power-law scalings between performance and dataset size. More recent work also investigated scaling between model size and data

size; their work is perhaps the closest to ours in the literature. Note, however, that one of them found super-linear scaling of dataset size with model size, whereas we find a sub-linear scaling.

幂律可以来自多种多样的来源。密度估计以及随机森林模型中模型与数据集规模的幂律缩放,可能与本文结果相关。这些模型表明,幂律指数可以被粗略地理解为数据中相关特征数的倒数。早期一些工作发现了性能与数据集规模之间的幂律缩放;近期的一些工作研究了模型规模与数据规模之间的尺度关系,这可能是文献中与我们最接近的工作。不过,需要指出的是,其中一项工作发现数据集规模相对模型规模呈超线性缩放,而我们发现的是次线性缩放。

There are some parallels between our findings on optimal allocation of compute and other recent work, including power-law learning curves. EfficientNets also appear to obey an approximate power-law relation between accuracy and model size. Very recent work studies scaling with both dataset size and model size for a variety of datasets, and fits an ansatz similar to ours. EfficientNet advocates scaling depth and width exponentially (with different coefficients) for optimal performance of image models, resulting in a power-law scaling of width as a function of depth. We find that for language models this power should be roughly one when scaling up (as width/depth should remain fixed). But more importantly, we find that the precise architectural hyperparameters are unimportant compared to the overall scale of the language model. In prior work it was argued that deep models can function as ensembles of shallower models, which could potentially explain this finding.

我们在算力最优分配方面的发现与近期其它工作(包括幂律学习曲线)存在一定的相似性。EfficientNet 似乎也服从一个关于准确率与模型规模的近似幂律关系。非常近期的工作针对多种数据集研究了模型与数据规模的联合缩放,并拟合了一个与我们类似的假设。EfficientNet 主张对图像模型以指数方式(不同系数)同时缩放深度与宽度,从而得到宽度作为深度函数的幂律关系。我们发现,对于语言模型,在扩展时这个幂指数应大致为 1(即宽度/深度保持不变)。但更重要的是,我们发现精确的架构超参数与语言模型的总尺度相比并不重要。此前有工作认为,深度模型可以看作是若干浅层模型的集成,这或许能解释本文的发现。

Some studies fix computation per data example, which tends to scale in proportion to the number of model parameters, whereas we investigate scaling with both model size and the quantity of training computation. Various works have investigated generalization in highly overparameterized models, finding a “jamming transition” when the model size reaches the dataset size (this may require training many orders of magnitude beyond typical practice, and in particular does not use early stopping). We do not observe such a transition, and find that the necessary training data scales sublinearly in the model size. Expansions in the model size, particularly at large width, may provide a useful framework for thinking about some of our scaling relations. Our results on optimization, such as the shape of learning curves, can likely be explained using a noisy quadratic model, which can provide quite accurate predictions in realistic settings. Making this connection quantitative will require a characterization of the Hessian spectrum.

一些研究固定每个数据样本的计算量,其规模往往与模型参数量成正比;而我们同时考察了模型规模与训练计算量两者的缩放。已有若干工作研究了高度过参数化模型的泛化,发现当模型规模达到数据集规模时会出现堵塞相变(jamming transition)(这可能要比通常做法多若干数量级的训练步,且并未使用提前停止)。我们并未观察到这一相变,并发现所需的训练数据按模型规模呈次线性增长。模型规模上的展开,特别是大宽度下的展开,可能为理解我们的某些尺度关系提供一个有用的框架。我们关于优化的结果(例如学习曲线的形状)很可能可以用带噪二次模型来解释,该模型能在现实设置下给出相当准确的预测。把这层联系变得定量,需要对 Hessian 谱进行刻画。

8. Discussion

讨论

We have observed consistent scalings of language model log-likelihood loss with non-embedding parameter count N , dataset size D , and optimized training computation $C_{\min}$, as encapsulated in the $L(N, D)$ and $L(N, S)$ equations. Conversely, we find very weak dependence on many architectural and optimization hyperparameters. Since scalings with N , D , $C_{\min}$ are power-laws, there are diminishing returns with increasing scale.

我们已经观察到语言模型对数似然损失与非嵌入参数量 N 、数据集规模 D 以及优化后的训练算力 $C_{\min}$ 之间存在一致的尺度关系,这些关系被 $L(N, D)$ 与 $L(N, S)$ 公式所概括。反过来,我们发现性能对许多架构与优化超参数的依赖非常弱。由于 N 、 D 、 $C_{\min}$

的缩放都是幂律,因此随着尺度增大,回报会递减。

We were able to precisely model the dependence of the loss on N and D , and alternatively on N and S , when these parameters are varied simultaneously. We used these relations to derive the compute scaling, magnitude of overfitting, early stopping step, and data requirements when training large language models. So our scaling relations go beyond mere observation to provide a predictive framework. One might interpret these relations as analogues of the ideal gas law, which relates the macroscopic properties of a gas in a universal way, independent of most of the details of its microscopic constituents.

我们能够精确刻画同时变化 N 与 D (以及 N 与 S)时损失对它们的依赖。我们利用这些关系,推导了训练大语言模型时的算力缩放、过拟合程度、提前停止的步数以及数据需求。因此,本文给出的尺度关系不只是简单的观察,而是提供了一个可预测的框架。我们可以把这种关系类比为理想气体定律:它以一种普适的方式刻画气体的宏观性质,与其微观组分的多数细节无关。

It is natural to conjecture that the scaling relations will apply to other generative modeling tasks with a maximum likelihood loss, and perhaps in other settings as well. To this purpose, it will be interesting to test these relations on other domains, such as images, audio, and video models, and perhaps also for random network distillation. At this point we do not know which of our results depend on the structure of natural language data, and which are universal. It would also be exciting to find a theoretical framework from which the scaling relations can be derived: a “statistical mechanics” underlying the “thermodynamics” we have observed. Such a theory might make it possible to derive other more precise predictions, and provide a systematic understanding of the limitations of the scaling laws.

很自然地猜想:这些尺度关系也将适用于其它以最大似然损失进行训练的生成式建模任务,甚至可能适用于更广泛的场景。为此,将这些关系在图像、音频、视频等其它领域(以及随机网络蒸馏)上进行检验是很有意义的。目前,我们尚不清楚本文的哪些结果依赖于自然语言数据的结构,哪些具有普适性。同样令人振奋的是,若能找到一个理论框架,从这个框架中推导出这些尺度关系——也就是为本文观察到的热力学提供底层的统计力学——那就更好了。这样的理论也许能让我们推导出其它更精确的预测,并对我们的尺度定律的局限性提供系统性的理解。

In the domain of natural language, it will be important to investigate whether continued improvement on the loss translates into improvement on relevant language tasks. Smooth quantitative change can mask major qualitative improvements: “more is different”. For example, the smooth aggregate growth of the economy provides no indication of the specific technological developments that underwrite it. Similarly, the smooth improvements in language model loss may hide seemingly qualitative changes in capability.

在自然语言领域,值得研究的问题是:损失上的持续改进是否能转化为相关语言任务上的改进。平滑的、定量的变化可能掩盖重大的、定性的变化:多即不同。例如,经济总量的平滑增长无法预示背后支撑它的具体技术进步;类似地,语言模型损失的平滑提升也可能掩盖看似质的能力变化。

Our results strongly suggest that larger models will continue to perform better, and will also be much more sample efficient than has been previously appreciated. Big models may be more important than big data. In this context, further investigation into model parallelism is warranted. Deep models can be trained using pipelining, which splits parameters depth-wise between devices, but eventually requires increased batch sizes as more devices are used. Wide networks on the other hand are more amenable to parallelization, since large layers can be split between multiple workers with less serial dependency. Sparsity or branching (e.g. Inception) may allow for even faster training of large networks through increased model parallelism. And using methods that grow networks as they train, it might be possible to remain on the compute-efficient frontier for an entire training run.

本文的结果强烈表明:更大的模型将继续表现更好,而且比此前所认识到的更加样本高效。大模型可能比大数据更重要。在此背景下,对模型并行的进一步研究是必要的。深度模型可以采用流水(pipelining)方式训练——将参数按深度切分到不同设备上,但在使用更多设备时最终仍需要增大批大小。宽网络则更易于并行化,因为大型层可以被拆分到多个 worker 上,且串行依赖更少。稀疏化或分支结构(如 Inception)或许能通过更高的模型并行度,让大网络的训练更快。而采用让网络在训练过程中生长的方法,有可能在整轮训练中始终保持在算力高效的前沿上。

Acknowledgements

致谢

We would like to thank Shan Carter, Paul Christiano, Jack Clark, Ajeya Cotra, Ethan Dyer, Jason Eisner, Danny Hernandez, Jacob Hilton, Brice Menard, Chris Olah, and Ilya Sutskever for discussions and for feedback on drafts of this work.

我们感谢 Shan Carter、Paul Christiano、Jack Clark、Ajeya Cotra、Ethan Dyer、Jason Eisner、Danny Hernandez、Jacob Hilton、Brice Menard、Chris Olah 与 Ilya Sutskever 在本文撰写过程中提供的讨论与对初稿的反馈。

Appendix A. Summary of Power Laws

附录 A. 幂律汇总

For easier reference, we provide a summary of the key trends described throughout the paper.

为便于查阅,本节汇总了文中所述的关键趋势。

Parameters / ■■	Data / ■■	Compute / ■■	Batch Size / ■■■	Equation / ■■
N	∞	∞	Fixed	$L(N) = (N_c / N)^{\alpha_N}$
∞	D	Early stop	Fixed	$L(D) = (D_c / D)^{\alpha_D}$
Optimal	∞	C	Fixed	$L(C) = (C_c / C)^{\alpha_C}$ (naive)
N_opt	D_opt	C_min	$B \ll B_{crit}$	$L(C_{min}) = (C_c^{min} / C_{min})^{\alpha_{C^{min}}}$
N	D	Early stop	Fixed	$L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$
N	∞	S steps	$B \ll B_{crit}$	$L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{min}(S, B))^{\alpha_S}$

Table 2 — Key trend equations (summary).

■ 2 — ■■■■■■■■■■

The empirical fitted values for these trends are:

这些趋势的经验拟合值为:

Power Law / ■■	Exponent / ■■	Scale (tokenization-dependent) / ■■(■■■■)
$\alpha_N = 0.076$	—	$N_c = 8.8 \times 10^{13}$ params (non-embed)
$\alpha_D = 0.095$	—	$D_c = 5.4 \times 10^{13}$ tokens
$\alpha_C = 0.057$	—	$C_c = 1.6 \times 10^4$ PF-days
$\alpha_{C^{min}} = 0.050$	—	$C_c^{min} = 3.1 \times 10^4$ PF-days
$\alpha_B = 0.21$	—	$B_* = 2.1 \times 10^4$ tokens
$\alpha_S = 0.76$	—	$S_c = 2.1 \times 10^3$ steps

Table 3 — Key parameters to trend fits.

■ 3 — ■■■■■■■■■■

The optimal parameters for compute efficient training are given by:

算力高效训练的最优参数为:

Compute-Efficient Value / ■■■■■	Power Law / ■■	Scale / ■■
$N_{opt} = N_e \cdot C_{min}^{p_N}$	$p_N = 0.73$	$N_e = 1.3 \times 10^4$ params
$B \ll B_{crit} = B_* / L^{1/\alpha_B} = B_e \cdot C_{min}^{p_B}$	$p_B = 0.24$	$B_e = 2.0 \times 10^4$ tokens
$S_{min} = S_e \cdot C_{min}^{p_S}$ (lower bound)	$p_S = 0.03$	$S_e = 5.4 \times 10^3$ steps
$D_{opt} = D_e \cdot C_{min}^{p_D}$ (1 epoch)	$p_D = 0.27$	$D_e = 2 \times 10^4$ tokens

Table 4 — Trends for compute-efficient training.

■ 4 — ■■■■■■■■■■

Appendix B. Empirical Model of the Compute-Efficient Frontier

附录 B. 算力高效前沿的经验模型

In this appendix we derive the optimal performance, model size, and number of training steps as a function of the compute budget, starting from $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S)^{\alpha_S}$. Here S represents the number of parameter updates when training at the critical batch size $B(L) = B_* / L^{1/\alpha_B}$. We would like to determine optimal training parameters for a fixed compute budget, so we replace $S = C / (6 N B(L))$, where C is the number of FLOPs used in the training run. Setting $\partial_N L|_C = 0$ yields the condition for optimality:

本附录中,我们从 $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S)^{\alpha_S}$

出发,推导损失、模型规模、训练步数随算力预算的最优变化关系。其中 S 是在临界批大小 $B(L) = B_* / L^{1/\alpha_B}$ 下训练时的参数更新次数。为确定固定算力预算下的最优训练参数,我们将 S 替换为 $C / (6 N B(L))$,其中 C 是该训练运行所消耗的 FLOPs 数。令 $\partial_N L|_C = 0$,得到最优性条件:

$$\begin{aligned}(\alpha_N / \alpha_S) \cdot (N_c / N)^{\alpha_N} &= (6 B_* S_c \cdot N / (L^{1/\alpha_B} C))^{\alpha_S} \\ (\alpha_N / \alpha_S) \cdot (N_c / N)^{\alpha_N} &= (6 B_* S_c \cdot N / (L^{1/\alpha_B} C))^{\alpha_S}\end{aligned}$$

Combining these equations, for compute-efficient training we should train to a fixed percentage $\alpha_N / \alpha_S \approx 10\%$ above the converged loss. Eliminating N yields a power-law dependence of performance on compute: $L(C) = (C_c / C)^{\alpha_C}$, where $\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052$ and $C_c = 6 N_c B_* S_c (1 + \alpha_N / \alpha_S)^{1/\alpha_S + 1/\alpha_N} \cdot (\alpha_S / \alpha_N)^{1/\alpha_S}$. Similarly, eliminating L yields $N(C) / N_c = (C / C_c)^{\alpha_C / \alpha_N} \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_N}$, and $S(C) = (C_c / (6 N_c B_*)) \cdot (1 + \alpha_N / \alpha_S)^{-1/\alpha_N} \cdot (C / C_c)^{\alpha_C / \alpha_S}$.

结合这些方程可知:在算力高效训练下,我们应当训练至固定比收敛损失高 $\alpha_N / \alpha_S \approx 10\%$ 的水平。消去 N ,得到性能关于算力的幂律依赖: $L(C) = (C_c / C)^{\alpha_C}$,其中 $\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052$,且 $C_c = 6 N_c B_* S_c \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_S + 1/\alpha_N} \cdot (\alpha_S / \alpha_N)^{1/\alpha_S}$ 。类似地,消去 L 得到 $N(C) / N_c = (C / C_c)^{\alpha_C / \alpha_N} \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_N}$,以及 $S(C) = (C_c / (6 N_c B_*)) \cdot (1 + \alpha_N / \alpha_S)^{-1/\alpha_N} \cdot (C / C_c)^{\alpha_C / \alpha_S}$ 。

Typically, researchers train models until they appear to be close to convergence. We compare the efficient training procedure to this more typical setup by defining a convergence factor f via $L(N, C) = (1 + f) \cdot L(N, \infty)$. For compute-efficient training we have $f = \alpha_N / \alpha_S \approx 10\%$, but researchers typically use a much smaller value, say $f' = 2\%$. For a fixed value of the loss, we predict $N_f / N_{f'} \approx 2.7$, $S_f / S_{f'} \approx 0.13$, $C_f / C_{f'} \approx 0.35$: compute-efficient training uses $7.7\times$ fewer parameter updates, $2.7\times$ more parameters, and 65% less compute to reach the same loss.

通常,研究者会将模型训练至接近收敛。我们通过定义收敛因子 f 来比较算力高效训练与

这一更通常的做法: $L(N, C) = (1 + f) \cdot L(N, \infty)$ 。对算力高效训练, $f = \alpha_N / \alpha_S \approx$

10% ;而研究者通常使用更小的值,例如 $f' = 2\%$ 。在固定损失值下,我们预测 $N_f / N_{f'} \approx 2.7, S_f / S_{f'} \approx 0.13, C_f / C_{f'} \approx 0.35$:算力高效训练达到同一损失,使用的参数更新次数少 7.7 倍,参数量多 2.7 倍,而算力节省 65% 。

Appendix C. Caveats

附录 C. 注意事项

We list some potential caveats to our analysis:

我们在本节列出对本文分析的一些潜在注意事项:

- At present we do not have a solid theoretical understanding for any of our proposed scaling laws. The scaling relations with model size and compute are especially mysterious. It may be possible to understand scaling at very large D holding model size fixed, and also the shape of learning curves late in training, by modeling the loss with a noisy quadratic. But the scaling with D at very large model size still remains mysterious. Without a theory or a systematic understanding of the corrections to our scaling laws, it's difficult to determine in what circumstances they can be trusted.

- 目前,我们对所提出的任意一条尺度定律都没有坚实的理论理解。与模型规模、算力相关的尺度关系尤其令人费解。在固定模型规模的情况下,通过用带噪二次模型来建模损失,或许可以理解极大 D 处的尺度以及训练后期的学习曲线形状。但在极大模型规模下, D 的尺度关系仍然神秘莫测。缺乏理论或对尺度定律修正项的系统理解,就很难判断其在何种情况下仍可被信赖。

- We are not especially confident in the prediction of $B_{\text{crit}}(L)$ for values of the loss far outside the range we have explored. Changes in B_{crit} could have a significant impact on trade-offs between data parallelism and the number of serial training steps required, which would have a major impact on training time.
 - 对于远在我们所考察的损失范围之外的情况,我们对 $B_{\text{crit}}(L)$ 的预测并不十分有把握。 B_{crit} 的变化可能会显著影响数据并行与所需串行步数之间的权衡,从而对训练时间产生重大影响。
- We did not thoroughly investigate the small data regime, and our fits for $L(N, D)$ were poor for the smallest values of D (where an epoch corresponded to only 40 steps). Furthermore, we did not experiment with regularization and data augmentation. Improvements in these could alter our results, quantitatively or qualitatively.
 - 我们并未深入考察小数据模式,在 D 取最小值时(此时一个 epoch 仅对应 40 步), $L(N, D)$ 的拟合较差。此外,我们也未尝试正则化与数据增强方面的实验。这些方面的改进可能在定量乃至定性上改变我们的结果。
- We used the estimated training compute $C \approx 6 N B S$, which did not include contributions proportional to n_{ctx} (see Section 2.1). So our scalings with compute may be confounded in practice in the regime of very large n_{ctx} , specifically where $n_{\text{ctx}} \gg 12 d_{\text{model}}$.
 - 我们使用的是训练算力的估计 $C \approx 6 N B S$,其中未包含与 n_{ctx} 成正比的项(参见第 2.1 节)。因此,在 n_{ctx} 非常大的情形(特别是 $n_{\text{ctx}} \gg 12 d_{\text{model}}$ 时)中,我们对算力的尺度关系在实践中可能受到混淆。
- We tuned learning rates, and we experimented with learning rate schedules. But we may have neglected to tune some hyperparameter (e.g. initialization scale or momentum) that has an important effect on scaling.
 - 我们对学习率进行了调参,并尝试了多种学习率调度。但我们可能忽略了某些对尺度关系有重要影响的超参数(如初始化尺度或动量)的调参。
- The optimal choice of learning rate is sensitive to the target loss. When training close to convergence, it may be necessary to use a smaller learning rate to avoid divergences. But when conducting a short training run (e.g. due to compute limitations), it may be possible to use a larger learning rate. We did not experiment with higher learning rates for training runs that did not proceed to convergence.
 - 最优学习率的选择对目标损失敏感。当训练接近收敛时,可能需要使用更小的学习率以避免发散;而当进行短时间训练(例如受算力限制)时,则可以使用更大的学习率。我们并未对那些未训练至收敛的运行尝试更高的学习率。

Note: This is a faithful, full-text translation of the main body of the paper, plus the key appendix tables (Tables 2–4 of the Appendix). Figures have been referenced by number but not re-rendered; please consult the original paper for figures. Mathematical notation is linearized for PDF rendering (e.g., C_{min} , α_N , $D \sim C^{0.27}$).